



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Studio e sviluppo di un sistema di Intrusion Detection in ambiente IoT mediante tecniche di Machine learning

RELATORE

Prof. Fabio Palomba

Università degli Studi di Salerno

CANDIDATO

Francesco Petrillo

Matricola: 0512116329

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Amare e vincere è la cosa migliore, amare e perdere quella immediatamente successiva

-William Thackery

Abstract

Un IDS è *uno strumento di sicurezza della rete che monitora il traffico di rete e i dispositivi alla ricerca di attività dannose note, attività sospette o violazioni delle politiche di sicurezza*. In questo lavoro è affrontato lo sviluppo di un IDS specifico per ambienti IoT, particolarmente vulnerabili ad attacchi informatici. Nello stato dell'arte attuale sono disponibili molti lavori riguardo gli IDS in IoT, ma quasi nessuno entra nel dettaglio del loro sviluppo, dando informazioni sui dataset disponibili o i modelli più indicati. La tesi si propone di fare proprio questo, raccontare del processo creativo che c'è dietro gli IDS in IoT, descrivendo man mano le criticità riscontrate e come queste ultime possono fungere da spunto di riflessione per migliorare l'attuale stato dell'arte.

Dopo aver addestrato 4 modelli diversi sul dataset BoT-IoT opportunamente preprocessato, sono stati ottenuti risultati più che positivi, paragonabili a quelli disponibili nello stato dell'arte. Questi risultati tuttavia suggeriscono la necessità di introdurre nell'ambiente metriche di valutazione post-rilascio, che garantiscano l'effettiva efficacia di modelli che potrebbero non essere adeguati agli ambienti reali a causa di dataset con problemi di sbilanciamento e poco generalizzati.

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	v
1 Introduzione	1
1.1 Contesto applicativo	1
1.2 Motivazioni e obiettivi	2
1.3 Risultati ottenuti	3
1.4 Struttura della tesi	3
2 Background e Stato dell'arte	4
2.1 L'Internet of Things	4
2.2 Gli Intrusion Detection Systems	5
2.3 Il Machine Learning	9
2.3.1 Apprendimento supervisionato	9
2.3.2 Apprendimento non supervisionato	10
2.3.3 Reinforcement learning	10
2.4 Gli IDS nell'IOT	11
2.4.1 I dataset IoT	12
2.4.2 Metriche di validazione	14
2.5 Stato dell'arte degli IDS basati su Machine Learning per l'IoT	16

3	Sviluppo del sistema	19
3.1	Obiettivi dello studio	19
3.2	Studio sul dataset	20
3.2.1	Ottenimento dei dati	20
3.2.2	Analisi preliminare	21
3.2.3	Esplorazione del dataset	23
3.2.4	Data Cleaning	35
3.2.5	Data normalization	36
3.2.6	Data Balancing	43
3.2.7	Data Encoding	43
3.3	Creazione dei modelli	44
4	Analisi dei risultati	49
5	Conclusioni	52
	Bibliografia	55

Elenco delle figure

2.1	Tasso di crescita dei dispositivi connessi dal 2019. [1]	5
2.2	Esempio di sistema IoT[2]	6
2.3	Confronto degli attacchi IoT tra il 2017 e il 2018[3]	6
2.4	Confronto tra IDS, IPS e IDPS[4]	8
2.5	Differenza tra NIDS e HIDS[5]	8
2.6	Esempio di apprendimento supervisionato[6]	10
2.7	Esempio di apprendimento non supervisionato[7]	11
2.8	Esempio di reinforcement learning[8]	12
3.1	CV per feature per subset	29
3.2	Heatmap del coefficiente di correlazione	31
3.3	Distribuzione del coefficiente di correlazione	31
3.4	Feature importance calcolata con Decision Tree	33
3.5	Distribuzioni delle feature in scala normale e logaritmica (parte 1).	38
3.6	Distribuzioni delle feature in scala normale e logaritmica (parte 2).	39
3.7	Distribuzioni delle feature in scala normale e logaritmica (parte 3).	40
3.8	Distribuzione della feature in scala normale (diviso, parte 1).	40
3.9	Distribuzioni delle feature in scala normale (diviso, parte 2).	41
3.10	Distribuzioni delle feature in scala normale (diviso, parte 3).	42

4.1	F1 Score dei modelli allenati su dataset bilanciato	49
4.2	F1 Score dei modelli allenati su dataset completo	50

Elenco delle tabelle

2.1	Dataset rilevanti per la rilevazione delle intrusioni in ambienti IoT	14
2.2	Metriche di valutazione per sistemi di rilevamento intrusioni	15
2.3	Confronto tra recenti lavori IDS basati su ML/DL in ambito IoT[9]	18
3.1	Descrizione delle feature del dataset	21
3.2	Ipotesi sulle feature sconosciute	23
3.3	Percentuale di valori nulli per colonna	24
3.4	Statistiche descrittive subset di attacco ($n = 72.6$ milioni di record)	25
3.5	Statistiche descrittive subset normale ($m = 9504$ record)	26
3.6	Varianza delle feature per traffico di attacco, normale e complessivo	27
3.7	Coefficiente di variazione (CV) delle feature per set di dati	28
3.8	Coefficienti di correlazione forti	32
3.9	Importanza delle feature secondo il modello	33
3.10	Processo di <i>Feature Selection</i>	36
3.11	Prestazioni del modello Random Forest sui due dataset	45
3.12	Prestazioni del modello SVM sui due dataset	46
3.13	Prestazioni del modello XGBoost sui due dataset	46
3.14	Prestazioni MLPC sui due dataset	47
3.15	Prestazioni del Balanced Random Forest	48

CAPITOLO 1

Introduzione

1.1 Contesto applicativo

Fin dalla creazione del primo *computer* l'informatica non ha fatto altro che evolversi a una velocità mai vista prima, mossa da una società che ha sempre visto nel progresso tecnologico l'opportunità per una vita migliore, in cui grossi problemi del passato fossero a un *click* dalla soluzione nel presente.

Dalla necessità di estendere la natura sociale dell'uomo è poi nato l'*Internet*, che ha modificato irreparabilmente il mondo in cui viviamo.

Nato come semplice strumento di comunicazione, l'Internet si è ben presto dimostrato essenziale in moltissimi settori della società, dalla migrazione dei negozi fisici agli e-commerce a franchise di videogiochi dal valore di miliardi di euro. In un mondo sempre più connesso era solo questione di tempo prima che anche oggetti di tutti i giorni, come lavatrici, frigoriferi o una semplice lampadina venissero investiti dalla rivoluzione dell'Internet, dando vita a quello che oggi è conosciuto come *Internet of Things*.

L'Internet delle cose descrive un mondo in cui gli oggetti non lavorano più in solitaria alla funzione per cui sono stati creati, ma comunicano tra di loro per garantire il massimo dell'efficienza nell'ambiente in cui si trovano, aprendo le porte

a possibilità impensabili il secolo scorso, come può essere un orologio che misura in tempo reale i parametri vitali di chi lo indossa e allerta in automatico le autorità se rileva anomalie, oltre che mostrare l'ora.

1.2 Motivazioni e obiettivi

Con la nascita di Internet è nata inevitabilmente una nuova "branca" della criminalità, quella degli attacchi informatici, processi con cui malintenzionati sfruttano punti deboli di sistemi informatici per compiere attività illecite, come l'interruzione forzata di un servizio o il furto di dati sensibili (e non).

I dispositivi IoT, che per natura trattengono un'enorme quantità di dati e hanno potere computazionale limitato, sono oggi uno dei bersagli preferiti dai criminali informatici, accanimento che spinge i ricercatori a trovare sistemi di sicurezza efficacemente implementabili sui suddetti dispositivi.

Anche l'IoT quindi, come in passato era successo ai computer connessi a Internet, ha iniziato a porsi il problema di riconoscere gli attacchi informatici mentre si verificavano, in modo da poter mettere in campo contromisure efficaci e prevenire i danni. È qui che entrano in gioco gli *Intrusion Detection System* (IDS), software messi a difesa di uno o più dispositivi con il compito di intercettare e classificare attacchi in arrivo.

Lo sviluppo degli IDS per l'IoT è ancora oggi in evoluzione, e grazie all'introduzione del *machine learning* è più efficiente che mai, ma ancora imperfetto. Sono molte le motivazioni che rendono difficile la creazione di IDS efficienti, tra le più rilevanti la non standardizzazione dei protocolli tra dispositivi diversi e la loro limitata capacità di calcolo.

Una situazione così complessa si manifesta nello stato dell'arte con una grande quantità di lavori che esplorano approcci diversi al problema, quali più specifici e quali più generici, e nello specifico del machine learning con la creazione di *dataset* e *modelli* differenti alla ricerca di quelli che meglio si prestano all'intrusion detection.

Il lavoro proposto in questa tesi ha l'obiettivo di verificare quanto gli strumenti attualmente disponibili in rete siano utili allo sviluppo di un IDS per l'IoT ed eventualmente porre attenzione su problematiche non emerse negli altri studi sul-

l'argomento. Lo sviluppo seguirà le pratiche classiche di creazione di modelli di ML: raccolta e processing dei dati, allenamento del modello e analisi dei risultati.

1.3 Risultati ottenuti

Il progetto ha portato alla creazione di diversi modelli di ML allenati sul dataset BoT-IoT, un dataset ampiamente utilizzato in altri lavori di questo genere, e i risultati ottenuti su ognuno sono stati particolarmente promettenti. Dei valori di *accuracy*, *precision*, *recall* e *F1 score* prossimi a 1 hanno indicato modelli robusti ed efficienti, nonchè in linea con quelli riportati in altri lavori dello stato dell'arte.

Lo studio inoltre ha portato alla luce quelli che potrebbero essere dei limiti nell'ambito degli IDS in IoT riguardanti qualità dei dataset e verifica dell'efficacia dei modelli, che se non considerati potrebbero portare alla diffusione di software giudicati erroneamente efficaci.

1.4 Struttura della tesi

La tesi è divisa successivamente in questo modo:

1. **Capitolo 2** - in questo capitolo è data una breve spiegazione dei concetti di *Internet of Things*, *Intrusion Detection* e *Machine Learning*, seguita da una relazione sullo stato dell'arte degli IDS in IoT che usano tecniche di ML.
2. **Capitolo 3** - In questo capitolo viene mostrato il processo di sviluppo del sistema.
3. **Capitolo 4** - In questo capitolo sono analizzati i risultati ottenuti ed esplicitate le riflessioni da essi derivanti.
4. **Capitolo 5** - In questo capitolo conclusivo è fatto un riassunto della tesi, seguito da considerazioni finali e proposte per gli sviluppi futuri dell'argomento.

CAPITOLO 2

Background e Stato dell'arte

Il lavoro proposto in questa tesi si basa su tre concetti principali legati all'informatica, l'*Internet of Things*, gli *Intrusion Detection Systems* e il *Machine learning*. Una conoscenza ad alto livello di ognuno, fornita in questo capitolo insieme a una breve analisi dello stato dell'arte, è necessaria per la sua comprensione.

2.1 L'Internet of Things

Con l'acronimo IoT (*Internet of Things*) si fa riferimento a una rete fatta di oggetti di uso comune che vanno dalla lampadina *smart* all'automobile a guida autonoma. Questi semplici oggetti di tutti i giorni, tramite l'installazione di *sensori*, *attuatori* e *software* hanno oggi la capacità di connettersi ad Internet e scambiarsi informazioni in tempo reale[10].

Il motivo per una transizione a un mondo fortemente connesso tramite l'IoT è una maggiore efficienza degli strumenti analogici di cui l'uomo si è servito fino ad ora, che lavorando insieme invece che singolarmente possono aiutare a migliorare la nostra sicurezza, produttività e semplicità nel fare le cose[11].

Il fenomeno dell' IoT è in rapida crescita da quando è nato, e non dà cenno di rallentare. Dal 2021 ad oggi si è registrato un **raddoppio** dei dispositivi connessi, che

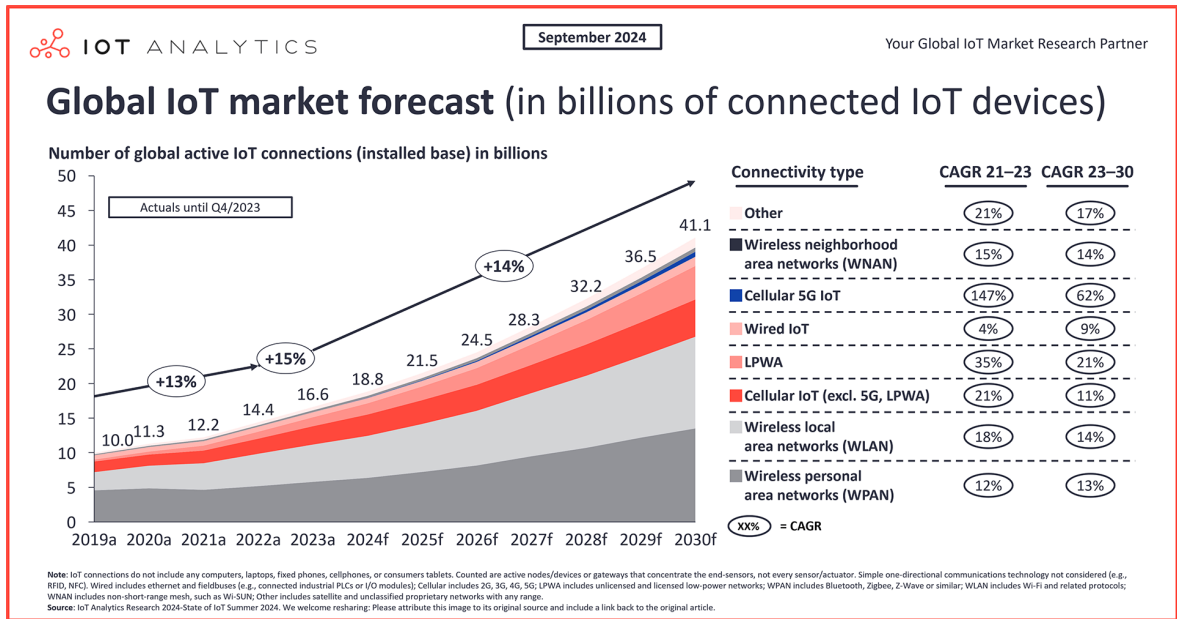


Figura 2.1: Tasso di crescita dei dispositivi connessi dal 2019. [1]

sono passati da 9 a 23 miliardi circa; stime affermano che entro la fine del decennio tale numero potrebbe raggiungere i 40 miliardi[12] (Fig. 2.1).

In genere un *sistema* IoT consiste in una serie di dispositivi comunicanti gestiti da una piattaforma centrale che ha il compito di controllare la comunicazione, analizzare e processare i dati, mantenere informazioni e verificare che i dispositivi si comportino nel modo giusto, oltre che individuare e risolvere eventuali vulnerabilità[13] (Fig. 2.2).

Per la sua natura eterogenea e mosso dalla sempre più rapida evoluzione tecnologica, il mondo degli IoT è in continuo cambiamento. Un'evoluzione tanto veloce rende necessario uno studio continuo sull'argomento, per permetterci di sfruttare al meglio le opportunità offerte da questa realtà, ma anche per essere pronti alle sfide che comporta, come problemi di sicurezza, privacy e comunicazione efficiente. Mano a mano che cresce la quantità di dispositivi IoT, infatti, aumenta anche la probabilità di attacchi informatici mirati (Fig. 2.3).

2.2 Gli Intrusion Detection Systems

Con la nascita di Internet è nata anche una nuova branca della sicurezza, la *cybersecurity*.

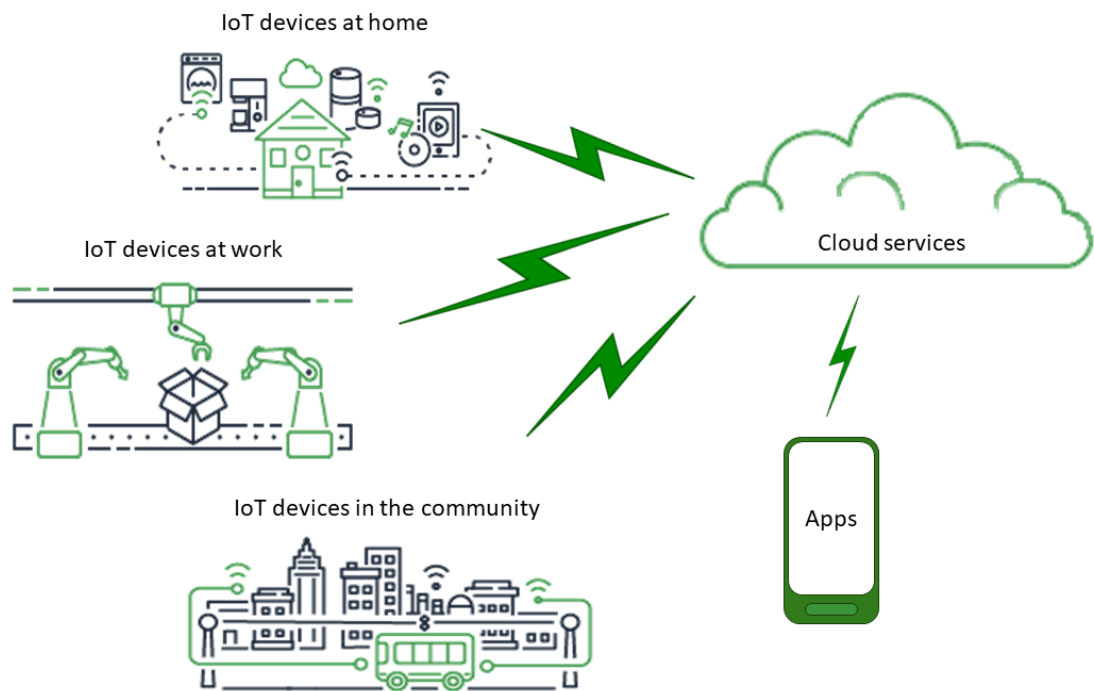


Figura 2.2: Esempio di sistema IoT[2]

IoT ATTACK VOLUME | YEAR-OVER-YEAR COMPARISON

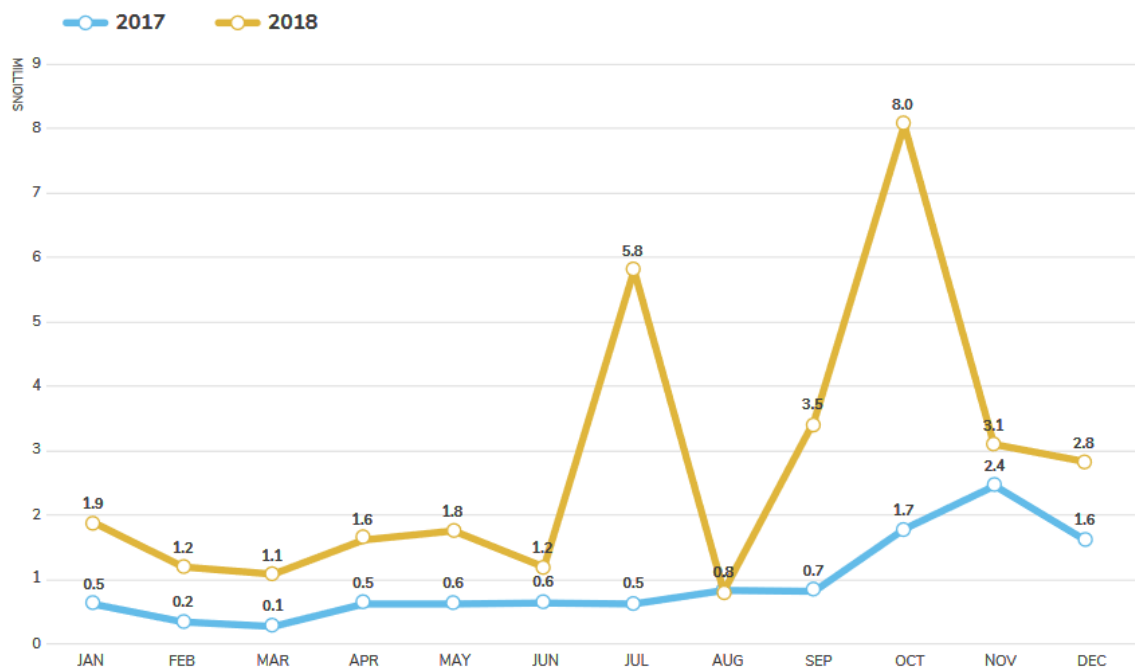


Figura 2.3: Confronto degli attacchi IoT tra il 2017 e il 2018[3]

Con il termine cybersecurity si intende “una qualsiasi tecnologia, pratica e politica per prevenire gli attacchi informatici o ridurne l’impatto”[14], dove si definisce attacco informatico “qualsiasi azione intenzionale che ha lo scopo di rubare, esporre, alterare, disabilitare o distruggere dati, applicazioni o altri asset tramite l’accesso non autorizzato a una rete, un sistema informatico o un dispositivo digitale”[15].

Detto in parole semplici, l’avvento dell’ Internet ha aperto le porte a nuovi tipi di crimini per cui è stato necessario creare nuove contromisure, come i comuni antivirus e software anti-malware.

In un ambiente pullulante di minacce, due paradigmi di difesa sono nati per contrastare gli attacchi: gli *Intrusion Detection Systems* (IDS) e gli *Intrusion Prevention Systems* (IPS). I primi sono dei software che automatizzano l’*intrusion detection*, ovvero il monitoraggio degli eventi che accadono all’interno di un computer o nella rete in cerca di minacce o pattern insoliti, mentre i secondi sono relativamente nuovi e fungono da evoluzione degli IDS (si potrebbero descrivere come IDS con la capacità di risolvere le minacce individuate)[16]. Come mostrato nella Fig. 2.4, l’unione dei due sistemi dà origine agli *IDPS*, un sistema ibrido che gode dei punti forti di entrambi.

Gli IDS possono essere suddivisi in due categorie, *host based* (HIDS) e *network based* (NIDS) (Fig. 2.5).

Un HIDS monitora le attività del dispositivo su cui è installato, come *system calls* o comandi *shell*, per identificare comportamenti sospetti e malevoli. Un sistema di questo tipo è estremamente specifico ed efficace ma è limitato a un’unica macchina.

Un NIDS invece opera a livello di *network*, analizzando il traffico di dati e ricavandone la probabilità di un attacco. Questo tipo di sistema è più flessibile nel riconoscere comportamenti illeciti e non richiede il sovraccarico di una singola macchina, ma è meno affidabile.

Un approccio ibrido, detto *Collaborative Intrusion Detection* (CID), punta a sfruttare i punti di forza di entrambi i sistemi per garantire robustezza[17].

Le metodologie più diffuse in letteratura per la creazione di IDS sono due, la *misuse detection*, che utilizza un database di attacchi noti per cercare e segnalare pattern malevoli conosciuti, e la *behaviour based detection*, che invece crea una linea di base per riconoscere il traffico “normale” e segnala qualsiasi anomalia[18].

Ancora una volta i due metodi presentano diversi punti di forza e debolezza,

IDS vs. IPS

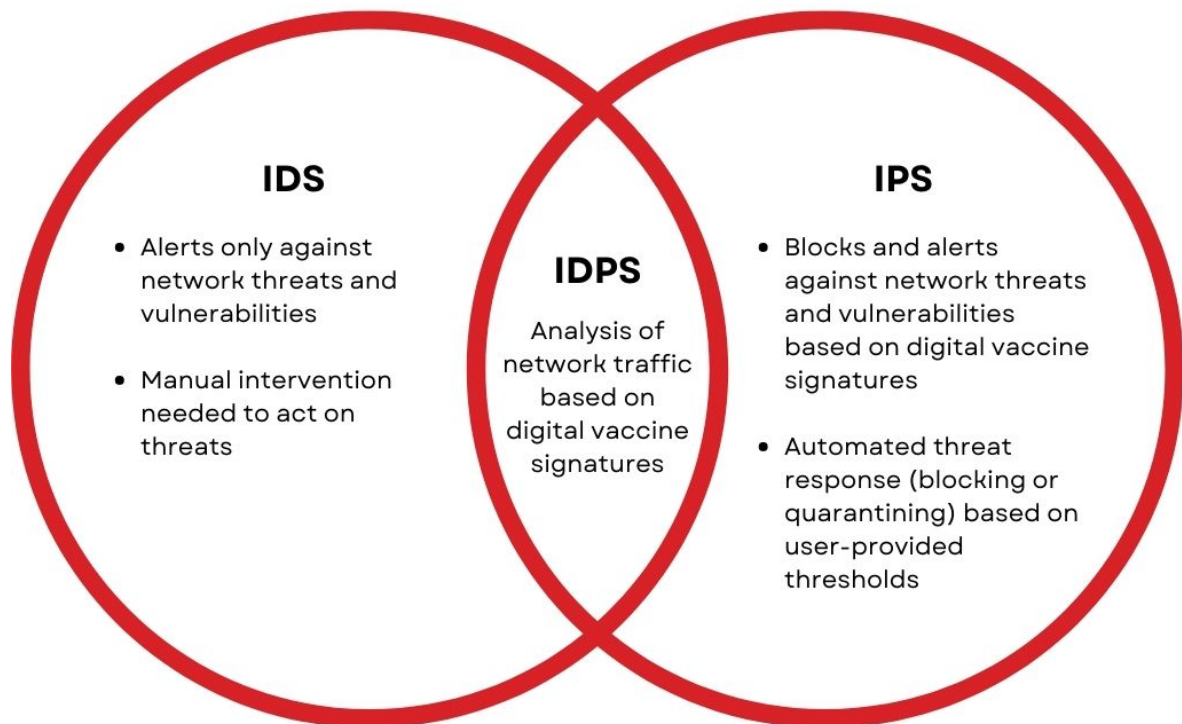
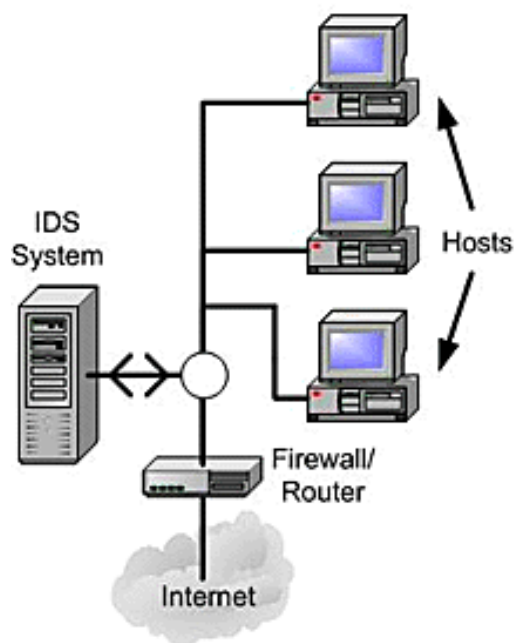


Figura 2.4: Confronto tra IDS, IPS e IDPS[4]

Network Based IDS



Host Based IDS

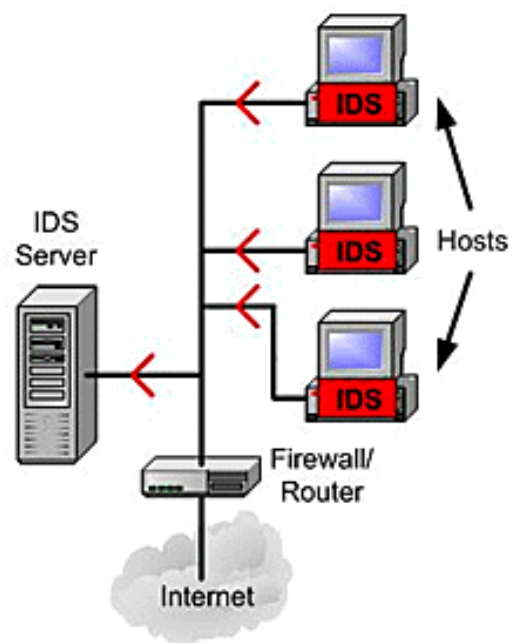


Figura 2.5: Differenza tra NIDS e HIDS[5]

il primo eccelle nel riconoscimento di attacchi conosciuti ma è poco robusto con i *zero-day attacks*, intrusioni che si basano su criteri nuovi e poco conosciuti, mentre il secondo è capace di identificare ogni tipo di attacco ma soffre di una grande quantità di falsi positivi (identifica sbagliando dei traffici normali come attacchi)[17].

Risulta evidente che il *machine learning* si presta particolarmente bene ad entrambi gli approcci, infatti oggi è alla base di tutti i principali sistemi di *intrusion detection*.

2.3 Il Machine Learning

L'intelligenza artificiale è la disciplina che studia se e in che modo si possano riprodurre i processi mentali più complessi mediante l'uso di un computer[19]. In pratica con IA s'intende l'insieme delle tecniche che permettano a un computer di *pensare* e *agire* umanamente e razionalmente. Se le prime dimostrazioni di intelligenza artificiale risalgono alla seconda metà '900 (un *breakthrough* importante è rappresentato dal *test di Turing*), con l'avanzare degli anni le tecniche di IA si sono andate sempre più ad affinare e a specializzare, creando di fatto una nuova branca dell'informatica.

Uno dei frutti principali di tale evoluzione è il *machine learning*, un insieme di tecniche e metodologie che sfruttano i **dati** per fare previsioni ed eseguire task senza la necessità di algoritmi specifici[20]. A seconda della relazione che hanno con i dati, tali tecniche possono essere raggruppate in tre grandi categorie:

- Apprendimento supervisionato
- Apprendimento non supervisionato
- Reinforcement learning

2.3.1 Apprendimento supervisionato

L'apprendimento supervisionato è un tipo di apprendimento che utilizza dati *etichettati* per allenare il modello (di machine learning). Con dati etichettati si intende esempi di istanze del problema che il modello è allenato a riconoscere associati a quella che viene definita *variabile dipendente*, ovvero l'esplicitazione della classe dell'istanza [21].

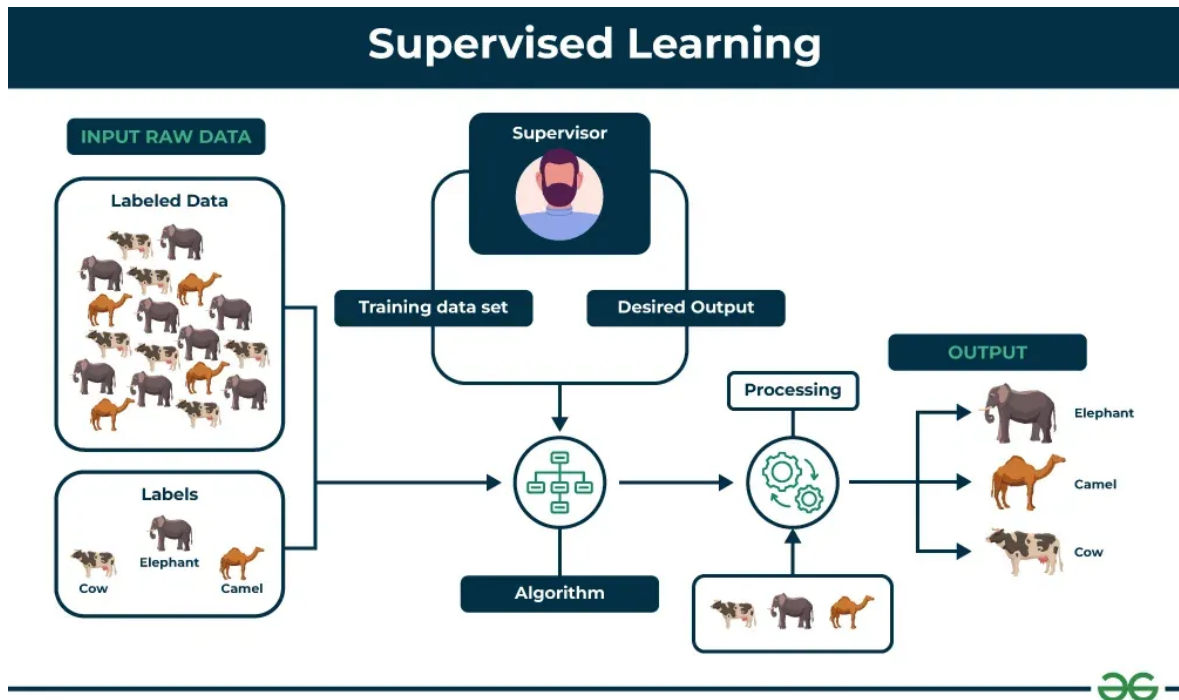


Figura 2.6: Esempio di apprendimento supervisionato[6]

Tipicamente modelli che apprendono in maniera supervisionata sfruttano la variabile dipendente dei dati conosciuti per creare un mapping tra input e output, che poi usano per predire l'output di dati non conosciuti [22] (Fig. 2.6).

2.3.2 Apprendimento non supervisionato

L'apprendimento si dice *non supervisionato* quando i dati da cui il modello impara non sono etichettati, quindi non c'è possibilità di imparare associazioni input-output e l'obiettivo diventa identificare pattern nascosti all'interno dei dati.

In genere algoritmi di questo tipo sono utilizzati per effettuare *clustering*, ovvero la suddivisione in gruppi dei dati con caratteristiche simili. Per fare un parallelo con l'apprendimento supervisionato, in questo caso è il modello a creare le etichette (Fig. 2.7).

2.3.3 Reinforcement learning

Il *reinforcement learning* è una tecnica di machine learning in cui un agente impara a fare le scelte giuste nell'interagire con l'ambiente tramite un sistema di *feedback*: se

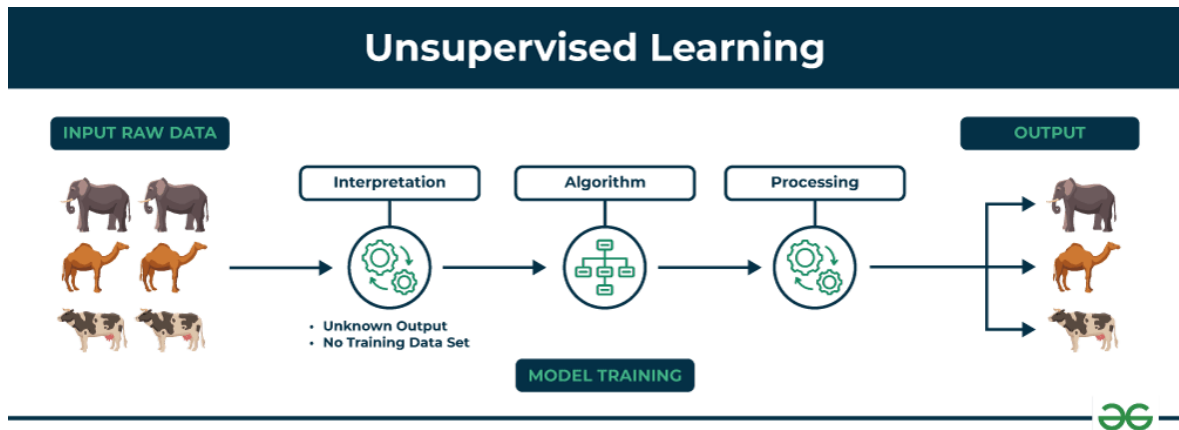


Figura 2.7: Esempio di apprendimento non supervisionato[7]

un'azione che compie è considerata "buona" (da chi crea il modello) esso viene ricompensato, se cattiva punito. In una situazione del genere il modello, con l'obiettivo di massimizzare le ricompense, imparerà a fare le scelte giuste. Per la loro natura algoritmi di questo tipo non sono particolarmente utilizzati per l'*analisi* dei dati, ma piuttosto per le interazioni delle macchine con l'ambiente (Fig. 2.8).

2.4 Gli IDS nell'IOT

Come già accennato nella sezione 2.1, l'IoT offre molte opportunità che vanno oltre l'Internet classico, ma comporta di conseguenza anche dei rischi più grandi.

Il fatto che tali sistemi per operare abbiano bisogno di mantenere grandi quantità di dati personali li rende il bersaglio ideale per attacchi informatici; le conseguenze in questi casi sono molteplici, spaziando dalla diffusione di dati sensibili all'interruzione di interi sistemi[23]. La sicurezza degli IoT ha appunto l'obiettivo di proteggere i dispositivi da questo tipo di minacce, ma un sistema di difesa robusto è difficile da implementare in questi ambienti a causa di risorse limitate, protocolli non standardizzati e mancanza di aggiornamenti[24].

Nel tempo sono state prese sempre più contromisure per garantire robustezza ai sistemi IoT, come cambiare le password di default, limitare l'accesso a internet dei dispositivi o aggiornare sistematicamente i software[23, 24]. Tra queste ovviamente si distinguono gli IDS e IPS, con e senza l'utilizzo del Machine Learning.

Reinforcement Learning in ML

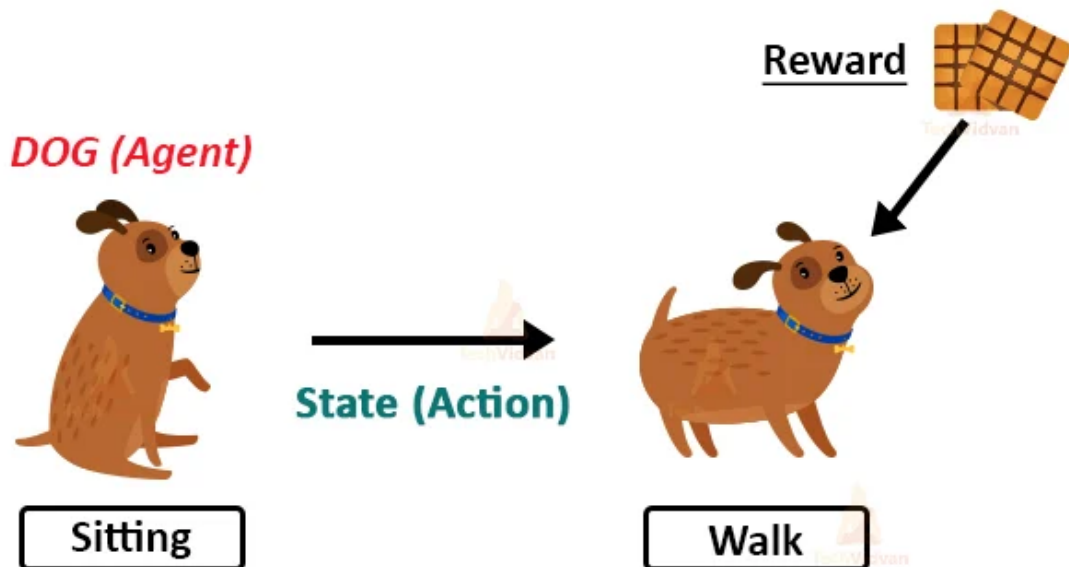


Figura 2.8: Esempio di reinforcement learning[8]

Ad oggi i sistemi di machine learning presenti in letteratura per gli IDS sono molti e diversi nelle tecniche utilizzate, e prima di analizzarne i più rilevanti è necessario fare una digressione sui *dataset* e sulle *tecniche di validazione* più comunemente utilizzati per questo genere di lavori.

2.4.1 I dataset IoT

Con *dataset* s'intende l'insieme di dati che vengono utilizzati per creare un modello di machine learning. Tipicamente durante questo processo un dataset viene diviso in due parti, il *training set* (l'80% circa) e il *test set* (il 20% circa), che servono rispettivamente all'allenamento e alla validazione del modello.

Un dataset per un ML-based IDS contiene record di utilizzo normale e anomalo di un sistema; quando si tratta di IoT in genere i dati rappresentano flussi di pacchetti a livello di network, log di sistema e sessioni[25].

I dataset disponibili per questo argomento sono molti e con caratteristiche differenti gli uni dagli altri (tipi di attacco, numero di record, tool utilizzati per la raccolta dati...). Tra i principali, come riassunto nella Tabella 2.1:

Bot-IoT Generato in laboratorio per simulare una botnet su dispositivi IoT, include attacchi DDoS/DoS, scansione e keylogging; contiene oltre 72 milioni di record TCP/UDP. [26, 27]

KDD-Cup99 Basato su dataset DARPA'98 raccolti da MIT Lincoln Labs, contiene circa 5 milioni di connessioni con 41 feature e attacchi suddivisi in DoS, Probe, R2L e U2R; storico benchmark per IDS. [28, 29]

CICIoT2023 Dataset del Canadian Institute for Cybersecurity: 105 dispositivi reali, 33 scenari d'attacco (es. Mirai), progettato per scenari IoT real-time. [30]

Edge-IIoTSet Testbed multilivello (cloud, fog, blockchain) con oltre 10 dispositivi industriali e attacchi come DoS, MiM, malware e injection. [31]

CoAP-DoS Concentrato su attacchi DoS sul protocollo CoAP, tipico degli ambienti IoT leggeri. [32]

CIC IoT-DIAD 2024 Include dati PCAP e flow CSV per l'identificazione del dispositivo e il rilevamento di anomalie in ambienti IoT. [33]

IDSIoT2024 Rete reale con 7 dispositivi, dataset disponibile in versioni bilanciate e sbilanciate; copre 12 categorie di attacchi. [34]

Gotham 2025 Testbed con 78 dispositivi su protocolli MQTT, CoAP e RTSP; include dati PCAP e CSV relativi a DDoS, brute-force, C&C e scansioni. [35]

TII-SSRC-23 Ampio traffico eterogeneo per valutazioni IDS, con baseline comparativa basata su ML/SVM. [36]

La natura eterogenea degli IoT rende quasi impossibile generare dataset "generici" efficaci, è anche per questo che buona parte dei ricercatori tende a creare le proprie banche dati, specifiche per il problema che intendono affrontare[25].

Dataset	Anno	Attacchi principali	Autori
Bot-IoT	2019	DDoS, DoS, scansione, key-logging	Koroniotis et al. [26], Moustafa [27]
KDD-Cup 99	1999	DoS, Probe, R2L, U2R	Stolfo et al. [28], Tavalae et al. [29]
CICIoT2023	2023	DDoS, Mirai, Brute-force, Recon, Web	Neto et al. [30]
Edge-IIoTSet	2022	DoS/DDoS, injection, malware, MiTM	Ferrag et al. [31]
CoAP-DoS	2022	DoS su protocollo CoAP	Mathews et al. [32]
CIC IoT-DIAD	2024	Device Identification, Anomaly Detection	Rabbani et al. [33]
IDSIoT2024	2025	12 categorie generiche di attacco	Koppula & Joseph [34]
Gotham2025	2025	DDoS, C&C, brute-force, scanning, CoAP	Belarbi et al. [35]
TII-SSRC-23	2023	Vari tipi di traffico anomalo	Herzalla et al. [36]

Tabella 2.1: Dataset rilevanti per la rilevazione delle intrusioni in ambienti IoT

2.4.2 Metriche di validazione

Per quantificare la "bontà" di un modello di machine learning, dopo la fase di *training* viene effettuata una fase di *testing*, grazie alla quale si riesce a dare dei punteggi alle caratteristiche di un modello, come il tasso di precisione con cui riconosce una determinata classe o quante volte fa predizioni errate.

In un ambiente tanto complesso come gli IDS in IoT, non esiste un indicatore unico che sia sufficiente a indicarne l'efficienza. Alcuni sostengono che il *Detection Rate* e il *False Alarm Rate* siano le metriche migliori per assestare la robustezza di un IDS[37], ma ultimamente l'importanza di una metrica su un'altra dipende da molti fattori, come il tipo di attacco da identificare, le risorse del dispositivo IoT o il

modello utilizzato[25].

Ci sono comunque alcune metriche applicate più spesso per la validazione degli IDS, elencate nella Tabella 2.2.

Tabella 2.2: Metriche di valutazione per sistemi di rilevamento intrusioni

Metrica	Formula	Descrizione
Accuratezza	$\frac{TP + TN}{TP + TN + FP + FN}$	Percentuale di predizioni corrette sul totale delle istanze.
Precisione (Tasso di rilevamento)	$\frac{TP}{TP + FP}$	Percentuale di positivi corretti rispetto a tutti i positivi previsti.
Sensibilità (Recall)	$\frac{TP}{TP + FN}$	Capacità del modello di identificare correttamente le istanze positive.
Specificità	$\frac{TN}{TN + FP}$	Capacità del modello di identificare correttamente le istanze negative.
F1-score	$\frac{2TP}{2TP + FP + FN}$	Media armonica tra precisione e sensibilità, utile per classi sbilanciate.
Tasso di falsi allarmi	$\frac{FP}{FN + FP}$	Percentuale di falsi allarmi sul totale degli allarmi errati.

Legenda delle abbreviazioni:

TP (True Positive): istanze positive classificate correttamente come positive

FP (False Positive): istanze negative classificate erroneamente come positive

TN (True Negative): istanze negative classificate correttamente come negative

FN (False Negative): istanze positive classificate erroneamente come negative

2.5 Stato dell’arte degli IDS basati su Machine Learning per l’IoT

Negli ultimi anni l’interesse verso l’impiego del machine learning all’interno dei sistemi di rilevamento delle intrusioni (IDS) per dispositivi IoT è cresciuto notevolmente. Diversi studi hanno proposto soluzioni sempre più raffinate, sfruttando algoritmi di apprendimento automatico per aumentare la precisione e ridurre i falsi positivi, adattandosi alle particolarità di ambienti fortemente eterogenei e vincolati come quello dell’Internet of Things (Tab. 2.3).

Un lavoro particolarmente interessante è quello di Xu et al. [38], che propone un sistema basato su tecniche di AutoML per classificare le intrusioni in modo completamente automatizzato. Utilizzando il dataset KDDcup99, gli autori sono riusciti a ottenere un’accuratezza del 99.7%, superando nettamente altri metodi più tradizionali.

In modo simile, Ngo et al. [39] hanno esplorato a fondo l’importanza della selezione e dell’estrazione delle caratteristiche, confrontando diverse combinazioni di modelli e tecniche. La loro analisi ha mostrato come l’estrazione delle feature sia particolarmente efficace con pochi parametri, e ha evidenziato che l’MLP si distingue sia nella selezione che nell’estrazione, offrendo ottime prestazioni complessive.

Viegas et al. [40], invece, si concentrano sulla validità delle metriche di valutazione nei sistemi IDS, e sottolineano l’importanza di utilizzare benchmark affidabili e variabili per una valutazione imparziale. Il loro studio, che coinvolge modelli come DNN, RF e SVM su dataset ben noti come CIC-IDS2017 e CSE-CIC-IDS2018, conferma il vantaggio delle reti neurali profonde in termini di accuratezza (99%), mentre Naïve Bayes si distingue per un tasso di falsi positivi particolarmente basso.

Altri autori hanno invece puntato su tecniche specifiche per migliorare la qualità del rilevamento. Zakariah et al. [41], ad esempio, propongono un campionamento sintetico adattivo basato su LSTM e CNN che si è dimostrato efficace nel classificare attacchi su NSL-KDD, con buoni valori di F1-score e AUC.

Gu e Lu [42] affrontano invece la problematica da un’altra prospettiva, integrando l’embedding delle feature basato su Naïve Bayes con una SVM, riuscendo così a

migliorare l'efficienza nella classificazione. Similmente, Guo et al. [43] hanno sperimentato con algoritmi evolutivi come il Chaos PSO per ottimizzare i parametri della SVM, ottenendo un'ottima capacità di rilevamento con un basso tasso di falsi allarmi.

In un'ottica più tradizionale, Kabir et al. [44] hanno dimostrato come anche modelli basati su SVM regolarizzati possano ancora offrire prestazioni elevate in termini di efficienza e accuratezza, se ben calibrati. Singh et al. [45] infine mostrano i vantaggi di una classificazione online tramite OS-ELM, con risultati promettenti sul dataset NSL-KDD e quello della Kyoto University.

Tabella 2.3: Confronto tra recenti lavori IDS basati su ML/DL in ambito IoT[9]

Autori	Anno	Metodologia proposta	Modelli ML/DL	Dataset	Prestazioni
Xu et al. [38]	2023	Metodo basato su AutoML per rilevamento intrusione e anomalie in IoT	AutoML	KDDcup99	Accuratezza del 99.7%, superiore ad altri algoritmi esistenti
Ngo et al. [39]	2023	Rilevamento intrusione basato su selezione ed estrazione delle feature	FS, FE, DT, RF, KNN, MLP, NB	UNSW-NB15	MLP eccelle sia in FS che FE; FE più affidabile con K piccolo
Viegas et al. [40]	2023	Schemi di valutazione affidabili per IDS di rete	DT, RF, SVM, NB, ANN, DNN	CIC-IDS2017, CSE-CIC-IDS2018, LUFlow	DNN ha l’accuratezza più alta (99%), NB il tasso FP più basso
Zakariah et al. [41]	2023	Campionamento sintetico adattivo per IDS	LSTM, CNN	NSL-KDD	F1 = 89% e 83%, AUC = 0.88 e 0.94, accuracy = 87%
Gu & Lu [42]	2021	Embedding feature NB con SVM per migliorare la qualità dei dati	SVM, Naïve Bayes	UNSW-NB15, CIDS2017	Alta accuratezza, basso false alarm rate
Guo et al. [43]	2016	Chaos PSO con SVM e programmazione lineare	SVM, MCLP	NSL-KDD	Alta detection, basso FPR rispetto a PSO/-CPSO
Kabir et al. [44]	2018	OA-LS-SVM: ottimizzazione support vector machine	LS-SVM	KDD 99	Buona accuratezza ed efficienza computazionale
Singh et al. [45]	2015	Profiling traffico con OS-ELM online	OS-ELM	NSL-KDD, Kyoto Benchmark	Riduzione FPR, maggiore accuratezza e velocità rilevamento

CAPITOLO 3

Sviluppo del sistema

Una volta fissati i concetti principali, possiamo procedere con lo sviluppo di un sistema vero e proprio. Il processo di creazione può essere diviso in tre grandi fasi: *studio del dataset*, in cui viene scelto e preprocessato il dataset che si userà nella fase successiva; *creazione dei modelli*, in cui uno o più modelli vengono allenati sul dataset risultante dalla fase precedente e *analisi dei risultati*, in cui si discutono le performance di ogni modello capendo quali siano più adatti ed evidenziando eventuali problemi comuni.

3.1 Obiettivi dello studio

L'obiettivo di questo studio è quello di creare un IDS efficiente specifico per l'IoT utilizzando tecniche di ML, per farlo si cercherà di giustificare ogni scelta effettuata in base alle domande:

- Come rendere il sistema specifico per l'IoT?
- Come migliorare l'efficienza del sistema?
- Come valutare l'efficacia del sistema?

In definitiva, il lavoro punta a rispondere alle seguenti *domande di ricerca*:

Domanda 1

Quali sono le prestazioni raggiungibili da modelli di Machine Learning per l’Intrusion Detection in ambiente IoT?

Domanda 2

Quali sono i punti critici dei sistemi così creati?

3.2 Studio sul dataset

Dovendo sviluppare un sistema specifico per l’IoT e in accordo con lo studio dello stato dell’arte si è deciso di utilizzare per questo lavoro il dataset Bot-IoT sviluppato nel 2019 da *Koroniotis et al.*[26].

I motivi di tale scelta sono molteplici; nonostante la presenza di dataset più recenti, infatti, *Bot-IoT* gode del fatto di essere stato ampiamente utilizzato in molti lavori di valore presenti nello stato dell’arte. Questo, unito alla sua natura prettamente IoT (tutti i dati raccolti derivano da dispositivi *smart*), lo rende il dataset perfetto per un tentativo di sviluppo di un IDS.

3.2.1 Ottenimento dei dati

Il dataset è pubblicamente accessibile in due formati, uno completo che conta circa 74 milioni di record e uno composto da solo il 5% dei dati, distribuito dagli stessi sviluppatori. Per questo lavoro si è deciso di utilizzare quello completo, eventualmente ridotto in dimensioni se necessario.

Il dataset è stato scaricato dal sito di *Kaggle*, una piattaforma di *data science* nella quale è disponibile un’enorme quantità di banche dati[46]. Per la sua grandezza il download è diviso in 74 file .csv con un milione di righe ciascuno, più un file con i nomi delle 35 feature, per un peso totale che si avvicina ai 16GB.

3.2.2 Analisi preliminare

La prima fase nel processo di analisi di un dataset è lo studio delle sue feature, di cui bisogna comprendere quantità e descrizione. Fortunatamente il significato delle feature, schematizzato nella tabella 3.1, è stato esplicitato dagli autori nel paper relativo al suo rilascio[26].

Tabella 3.1: Descrizione delle feature del dataset

Feature	Descrizione
pkSeqID	Identificatore della riga
Stime	Ora di inizio del record
flgs	Flag dello stato del flusso visti nelle transazioni
proto	Rappresentazione testuale dei protocolli di transazione nel flusso di rete
saddr	Indirizzo IP sorgente
sport	Porta sorgente
daddr	Indirizzo IP di destinazione
dport	Porta di destinazione
pkts	Numero totale di pacchetti nella transazione
bytes	Numero totale di byte nella transazione
state	Stato della transazione
ltime	Ora finale del record
seq	Numero di sequenza Argus
dur	Durata totale del record
mean	Durata media dei record aggregati
stddev	Deviazione standard dei record aggregati
smac	Indirizzo MAC sorgente

Continua nella pagina successiva

Feature	Descrizione
dmac	Indirizzo MAC di destinazione
sum	Durata totale dei record aggregati
min	Durata minima dei record aggregati
max	Durata massima dei record aggregati
soui	?
doui	?
sco	?
dco	?
spkts	Numero di pacchetti da sorgente a destinazione
dpkts	Numero di pacchetti da destinazione a sorgente
sbytes	Numero di byte da sorgente a destinazione
dbytes	Numero di byte da destinazione a sorgente
rate	Pacchetti totali al secondo nella transazione
srate	Pacchetti al secondo da sorgente a destinazione
drate	Pacchetti al secondo da destinazione a sorgente
attack	Etichetta di classe: 0 per traffico normale, 1 per traffico di attacco
category	Categoria del traffico
subcategory	Sottocategoria del traffico

Nonostante la ricerca, non è stato possibile reperire informazioni relative alle feature "soui", "doui", "sco" e "dco", ma dal contesto è stato possibile fare delle ipotesi riportate nella tabella 3.2. Se in un secondo momento risulterà necessario, ne verrà effettuato uno studio più approfondito.

Adesso che sappiamo in linea di massima che tipo di dati stiamo trattando, si può procedere con il *preprocessing*.

Feature	Descrizione Ipotesi
soui	Source Unique IPs: Numero di indirizzi IP unici contattati dalla sorgente durante una finestra temporale specifica.
doui	Destination Unique IPs: Numero di indirizzi IP unici che hanno contattato la destinazione durante una finestra temporale specifica.
sco	Source Connection Count: Numero di connessioni stabilite dalla sorgente in un determinato periodo.
dco	Destination Connection Count: Numero di connessioni ricevute dalla destinazione in un determinato periodo.

Tabella 3.2: Ipotesi sulle feature sconosciute

3.2.3 Esplorazione del dataset

Dopo aver analizzato il significato dei dati, una esplorazione del dataset ci può aiutare a studiare la natura delle sue feature, cercando caratteristiche interessanti per il processo di *feature selection*, ovvero la rimozione di feature poco importanti o che potrebbero minare l'efficacia di un modello allenato su questi dati.

Per farlo si è deciso di procedere per fasi, in ordine:

- Individuazione di valori nulli o mancanti
- Studio delle feature di natura numerica
 - Studio delle caratteristiche descrittive di ogni feature
 - Studio della correlazione tra le feature
 - Studio dell'importanza delle feature
- Studio delle feature di natura categorica

Individuazione di valori nulli o mancanti

Prima di effettuare uno studio vero e proprio delle feature, bisogna occuparsi dei valori nulli all'interno del dataset; è buona pratica eliminarli prima di addestrare un modello, dato che potrebbero minare il suo corretto funzionamento.

Lo studio, riportato nella tabella 3.3, ha rilevato che le colonne `soui`, `doui`, `sco`, `dco`, `smac` e `dmac` presentano il 100% di valori nulli, mentre tutte le altre ne hanno una quantità prossima o uguale a 0.

Colonna	Percentuale di valori nulli
<code>soui</code>	100%
<code>doui</code>	100%
<code>sco</code>	100%
<code>dco</code>	100%
<code>smac</code>	100%
<code>dmac</code>	100%
<code>sport</code>	<0.004%
<code>dport</code>	<0.004%
<code>subcategory</code>	<0.00001%
<code>rate</code>	<0.00001%
<code>dbytes</code>	<0.00001%
<code>sbytes</code>	<0.00001%
<code>category</code>	<0.00001%
<code>attack</code>	<0.00001%
<code>dpkts</code>	<0.00001%
<code>srate</code>	<0.00001%
<code>drate</code>	<0.00001%

Tabella 3.3: Percentuale di valori nulli per colonna

Studio delle feature numeriche

Per feature numeriche s'intende le feature che sono rappresentate tramite valori numerici (float, int...). Queste vengono studiate separatamente dalle feature categoriche (stringhe, oggetti...) mediante formule matematiche.

In una prima fase calcoleremo le caratteristiche descrittive principali per ogni feature al fine di comprenderne le distribuzioni e identificare eventuali pattern o outlier. In genere è buona pratica considerare per questo studio l'interezza del dataset, ma data la natura sbilanciata di BoT-IoT si è deciso di creare due subset da studiare separatamente, uno per i record di attacco e uno per quelli normali.

I risultati sono riportati rispettivamente nelle tabelle 3.4 e 3.5.

Feature	Mean	Std	Min	25%	50%	75%	Max
stime	1.53×10^9	1.91×10^5	1.53×10^9	1.53×10^9	1.53×10^9	1.53×10^9	1.53×10^9
pkts	6.32	23.02	1.00	4.00	7.00	14.00	1.48×10^5
bytes	567.26	1.51×10^5	60.00	420.00	600.00	960.00	8.19×10^8
ltime	1.53×10^9	1.91×10^5	1.53×10^9	1.53×10^9	1.53×10^9	1.53×10^9	1.53×10^9
dur	23.73	18.56	0.00	15.17	23.98	37.70	2771.49
mean	1.81	1.60	0.00	0.15	2.72	3.98	5.02
stddev	0.81	0.87	0.00	0.02	1.36	2.08	2.50
sum	6.03	5.86	0.00	0.31	8.29	13.99	410.23
min	0.86	1.53	0.00	0.00	0.00	3.74	5.00
max	2.73	2.06	0.00	0.22	4.29	4.87	7.60
spkts	6.06	13.27	0.00	4.00	6.00	11.00	8.28×10^4
dpkts	0.26	14.28	0.00	0.00	0.00	1.00	1.18×10^5
sbytes	528.24	7.40×10^4	0.00	420.00	600.00	924.00	4.75×10^8
dbytes	39.02	1.32×10^5	0.00	0.00	0.00	60.00	8.18×10^8
rate	192.47	5.38×10^3	0.00	0.17	0.26	1.87	1.33×10^6
srate	11.59	480.93	0.00	0.15	0.24	0.70	6.67×10^5
drate	9.00	406.71	0.00	0.00	0.00	0.00	5.00×10^5

Tabella 3.4: Statistiche descrittive subset di attacco ($n = 72.6$ milioni di record)

Feature	Mean	Std	Min	25%	50%	75%	Max
pkSeqID	1.02×10^7	1.85×10^7	1.00	1.11×10^6	1.61×10^6	3.70×10^6	7.34×10^7
stime	1.53×10^9	6.66×10^5	1.53×10^9	1.53×10^9	1.53×10^9	1.53×10^9	1.53×10^9
pkts	2136.58	12250.54	1.00	2.00	2.00	3754.00	322677.00
bytes	1.83×10^6	1.15×10^7	60.00	176.00	625.00	3.41×10^6	3.14×10^8
ltime	1.53×10^9	6.66×10^5	1.53×10^9	1.53×10^9	1.53×10^9	1.53×10^9	1.53×10^9
seq	30172.40	62432.58	1.00	3728.50	19353.50	53144.00	524315.00
dur	102.01	399.57	0.00	0.25	5.05	58.18	3536.09
mean	1.21	1.97	0.00	0.00	0.17	4.91	5.00
stddev	0.19	0.46	0.00	0.00	0.00	1.49	2.50
sum	63.34	308.88	0.00	0.00	0.17	45.67	3502.99
min	0.73	1.45	0.00	0.00	0.14	3.92	5.00
max	1.33	2.14	0.00	0.00	0.17	5.00	5.00
spkts	1565.62	8754.38	1.00	1.00	2.00	2803.25	234761.00
dpkts	570.96	4985.71	0.00	0.00	1.00	822.50	161338.00
sbytes	1.27×10^6	7.87×10^6	42.00	108.00	176.00	2.46×10^6	2.25×10^8
dbytes	559089.67	4.90×10^6	0.00	0.00	120.00	8.05×10^5	1.52×10^8
rate	853.37	8166.00	0.00	0.20	10.97	147.89	333333.31
srate	824.05	20024.64	0.00	0.00	0.20	79.79	1.00×10^6
drate	5.24	36.20	0.00	0.00	0.00	20.50	2178.65

Tabella 3.5: Statistiche descrittive subset normale (m = 9504 record)

Varianza

La varianza di un gruppo di valori indica quanto questi si discostano dal valore medio. Questo è un valore particolarmente interessante quando si parla di dati: una varianza bassa presuppone un raccoglimento dei dati in prossimità della media, lasciando supporre che questi risultino poco efficaci per un'eventuale classificazione.

La varianza si calcola dalla media tramite la formula (3.2.1):

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2 \quad (3.2.1)$$

- σ^2 : varianza della popolazione
- N : numero totale di elementi nella popolazione
- x_i : valore del singolo elemento
- μ : media della popolazione

La tabella 3.6 mostra il calcolo delle varianze per feature nei due subset e nel dataset intero.

Feature	Varianza Attack	Varianza Normal	Varianza Global
stime	3.66×10^{10}	4.43×10^{11}	3.67×10^{10}
pkts	5.30×10^2	1.50×10^8	2.08×10^4
bytes	2.29×10^{10}	1.33×10^{14}	4.08×10^{10}
ltime	3.66×10^{10}	4.43×10^{11}	3.67×10^{10}
dur	3.44×10^2	1.60×10^5	3.66×10^2
mean	2.56	3.89	2.56
stddev	0.76	0.21	0.76
sum	3.44×10^1	9.54×10^4	4.73×10^1
min	2.33	2.11	2.33
max	4.25	4.57	4.25
spkts	1.76×10^2	7.66×10^7	1.05×10^4
dpkts	2.04×10^2	2.49×10^7	3.50×10^3
sbytes	5.47×10^9	6.19×10^{13}	1.38×10^{10}
dbytes	1.73×10^{10}	2.41×10^{13}	2.05×10^{10}
rate	2.90×10^7	6.67×10^7	2.90×10^7
srate	2.31×10^5	4.01×10^8	2.84×10^5
drate	1.65×10^5	1.31×10^3	1.65×10^5

Tabella 3.6: Varianza delle feature per traffico di attacco, normale e complessivo

Per quanto informativa, in questo caso la varianza non ci permette di confrontare le feature una con l'altra a causa della grande differenza tra le loro scale, che arriva a diversi ordini di grandezza. Una soluzione a questo problema sarebbe quella di normalizzare e ripetere il calcolo, ma esiste un valore, detto *coefficiente di variazione*, che indica la dispersione dei dati in maniera adimensionale, permettendo appunto un confronto di scale diverse. Il coefficiente si calcola tramite *media* e *deviazione standard* tramite la formula (3.2.2):

$$CV = \frac{\sigma}{\mu} \quad (3.2.2)$$

Dove:

- CV: Coefficiente di variazione (Coefficient of Variation)
- σ : Deviazione standard della popolazione
- μ : Media della popolazione

I coefficienti di variazione calcolati per ogni set di dati sono riportati nella tabella 3.7.

Feature	CV Attack	CV Normal	CV Global	CV Min
stime	1.25×10^{-4}	4.36×10^{-4}	1.25×10^{-4}	1.25×10^{-4}
pkts	3.64	5.73	21.8	3.64
bytes	267	6.30	250	6.30
ltime	1.25×10^{-4}	4.36×10^{-4}	1.25×10^{-4}	1.25×10^{-4}
dur	0.782	3.92	0.806	0.782
mean	0.882	1.64	0.882	0.882
stddev	1.07	2.36	1.07	1.07
sum	0.973	4.88	1.14	0.973
min	1.78	1.99	1.78	1.78
max	0.754	1.61	0.754	0.754
spkts	2.19	5.59	16.4	2.19
dpkts	55.6	8.73	178	8.73
sbytes	140	6.18	169	6.18
dbytes	3375	8.77	1277	8.77
rate	28.0	9.57	28.0	9.57
srate	41.5	24.3	45.5	24.3
drate	45.2	6.91	45.2	6.91

Tabella 3.7: Coefficiente di variazione (CV) delle feature per set di dati

Adesso possiamo confrontare le feature tra di loro e cercare *pattern* utili. Per una migliore visualizzazione abbiamo creato uno *scatter plot* che metta a confronto i CV dei due subset, visualizzabile alla Fig. 3.1. Per una migliore comprensione dei dati,

Data theft, Reconnaissance. È stato deciso di utilizzare il *coefficiente di Pearson* per questo studio, che mostra la correlazione *lineare* tra le feature tramite la formula (3.2.3):

$$r = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} \quad (3.2.3)$$

Dove:

- $\text{cov}(X, Y)$ è la covarianza tra le variabili X e Y
- σ_X è la deviazione standard della variabile X
- σ_Y è la deviazione standard della variabile Y

Per una migliore visualizzazione dei risultati è stata utilizzata una *heatmap*, ovvero una matrice con le feature sia sulle colonne che sulle righe che assegna un valore a ogni coppia (il CC), e colora le celle con colori dal blu al rosso a seconda di quanto è alto tale valore (Fig. 3.2).

Dai risultati ottenuti possiamo calcolare la distribuzione dei CC, che ci darà un'idea del grado di correlazione generale del dataset. Per visualizzarla utilizzeremo un grafico a istogramma mostrato nella Fig. 3.3. Anche per questo è stato deciso un *threshold* empirico di 0.7, dopo il quale le feature sono considerate fortemente correlate (Tab. 3.8).

Feature importance

Per completare lo studio delle feature e soprattutto per aiutarci ad agire al meglio in base ai risultati precedenti, abbiamo calcolato come ultima metrica la *feature importance*, ovvero la capacità di ogni feature di individuare split nel dataset in base alla variabile dipendente. Per spiegarla in termini semplici, una feature con un basso livello d'importanza non assume valori molto diversi tra le diverse classi del dataset (nel nostro caso "attacco" e "normale"), mentre una con alta importanza assume gruppi di valori diversi a seconda della classe.

Per calcolare questo indice abbiamo usato un *decision tree* applicato al dataset ridotto creato per il calcolo del CC e come variabile dipendente abbiamo scelto la

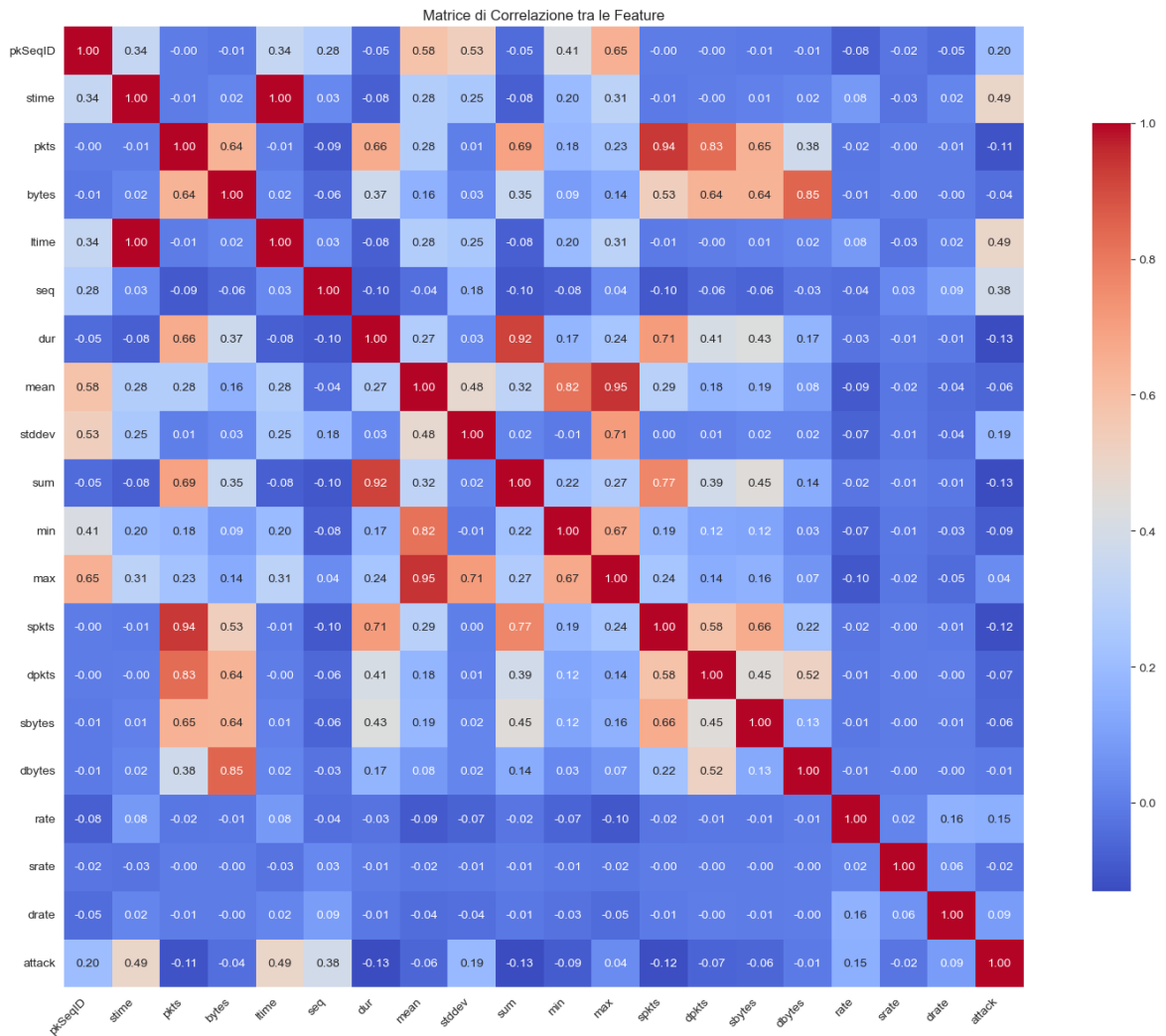


Figura 3.2: Heatmap del coefficiente di correlazione

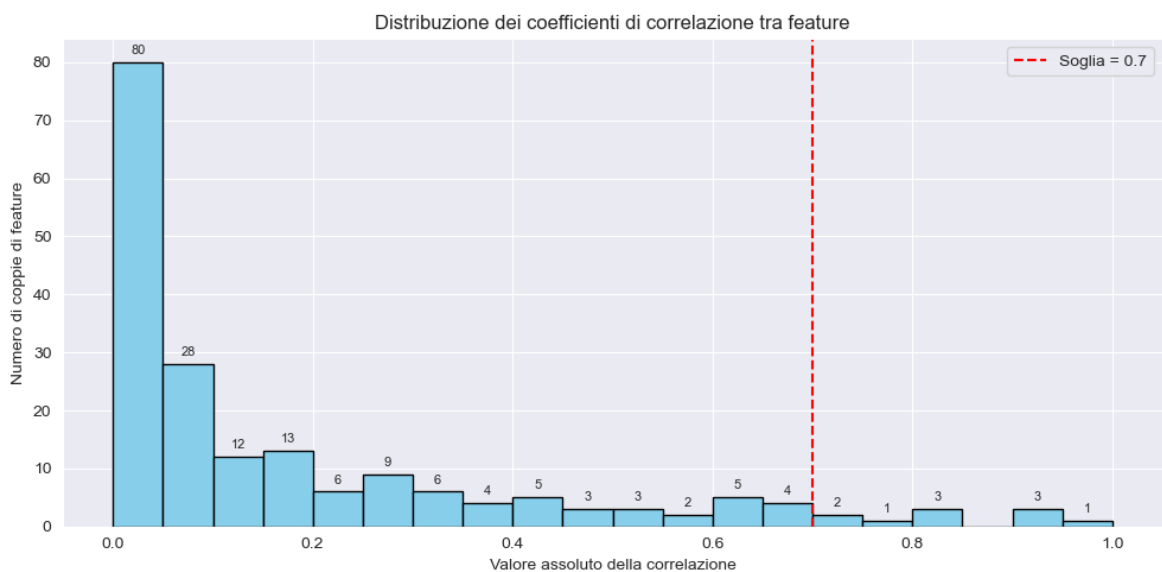


Figura 3.3: Distribuzione del coefficiente di correlazione

Feature 1	Feature 2	Coefficiente
ltime	stime	1.00
sum	dur	0.92
min	mean	0.82
max	mean	0.95
max	stddev	0.71
spkts	pkts	0.94
spkts	dur	0.71
spkts	sum	0.77
dpkts	pkts	0.83
dbytes	bytes	0.85

Tabella 3.8: Coefficienti di correlazione forti

feature category invece di attack, per evitare di assegnare a feature che riconoscono bene un tipo di attacco specifici valori di importanza bassi perché (ad esempio) vengono surclassate da altre più generiche.

Data la possibilità, in questa fase sono state analizzate anche le variabili categoriche, opportunamente soggette a *encoding* per poter essere utilizzate nell'albero decisionale.

Per visualizzare i risultati è stato creato un grafico a istogramma, mostrato nella Fig. 3.4, i risultati numerici sono invece riportati nella tabella 3.9.

Studio delle feature categoriche

Per lo studio delle feature categoriche in realtà non ci sono approcci matematici efficaci, lasciando spazio a una loro analisi *qualitativa* piuttosto che *quantitativa*. In questa sezione ci si propone di valutare, con l'ausilio della tabella esplicativa delle feature, l'effettiva rilevanza di ciascuna variabile categorica rispetto al problema di rilevamento delle intrusioni.

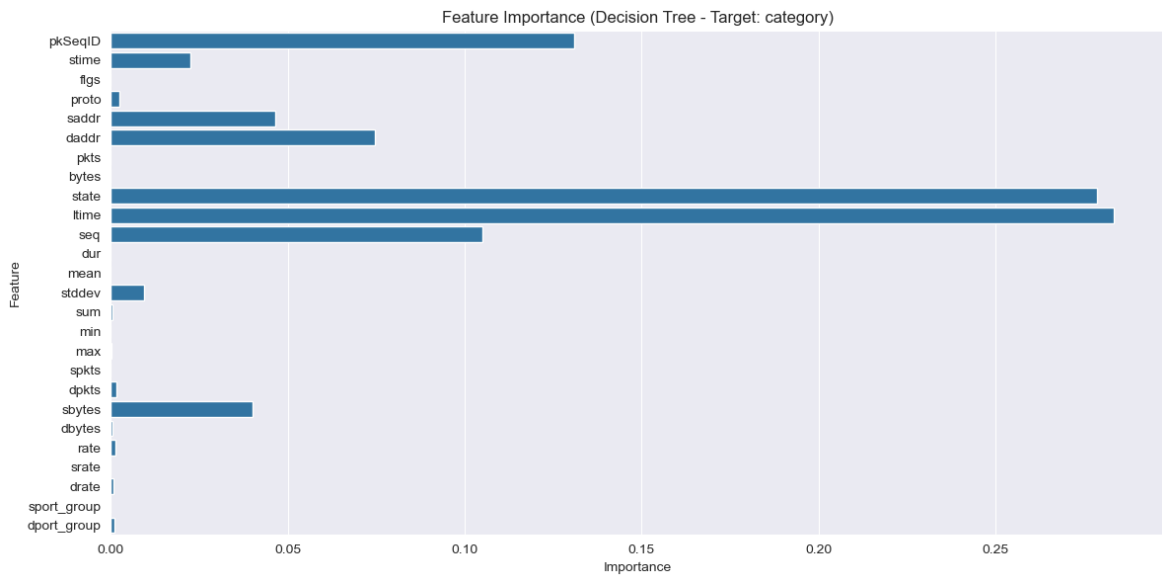


Figura 3.4: Feature importance calcolata con Decision Tree

Feature	Importanza
ltime	0.283445
state	0.278718
pkSeqID	0.130876
seq	0.104987
daddr	0.074694
saddr	0.046631
sbytes	0.040160
stime	0.022426
stddev	0.009506
proto	0.002428
dpkts	0.001592
rate	0.001372
dport_group	0.001034

Feature	Importanza
drate	0.000779
dbytes	0.000589
sum	0.000416
max	0.000346
flgs	0.000000
pkts	0.000000
bytes	0.000000
dur	0.000000
mean	0.000000
spkts	0.000000
min	0.000000
srate	0.000000
sport_group	0.000000

Tabella 3.9: Importanza delle feature secondo il modello

- **flgs**: rappresenta lo stato del flusso secondo il tool *Argus*, utilizzato per l'estrazione dei dati. In particolare, valori come *e* (indicante una ritrasmissione) potrebbero suggerire comportamenti anomali come tentativi di attacco DDoS, rendendola una feature potenzialmente utile.
- **proto**: identifica il protocollo di trasporto impiegato nella transazione. Sebbene nella maggior parte dei casi si tratti di TCP, UDP o ICMP, la presenza di protocolli meno comuni potrebbe essere sintomatica di attacchi mirati, ad esempio attacchi di tipo ARP poisoning.
- **saddr** e **daddr**: corrispondono rispettivamente agli indirizzi IP sorgente e destinazione. Nonostante la loro utilità sia limitata in modelli standard, potrebbero acquisire rilevanza in un contesto *time-series*, dove l'analisi del comportamento temporale delle comunicazioni può far emergere pattern sospetti.
- **sport** e **dport**: indicano le porte sorgente e destinazione della comunicazione. La loro elevata cardinalità rende difficile l'integrazione diretta nel modello; tuttavia, una loro categorizzazione in gruppi semantici (es. *well-known*, *registered*, *dynamic*) può ridurre la complessità e restituire informazioni significative sul tipo di servizio coinvolto.
- **state**: descrive lo stato della connessione (es. ESTABLISHED, CLOSED, ecc.). Alcuni stati specifici, come le connessioni parzialmente aperte (S0) o chiuse in modo anomalo, possono rappresentare indizi utili per rilevare tentativi di scansione o attacchi SYN flood.
- **category** e **subcategory**: definiscono rispettivamente la categoria e la sottocategoria del traffico. In fase di analisi possono offrire un prezioso punto di riferimento per la validazione dei risultati del modello. Tuttavia, essendo label derivate, non possono essere impiegate in fase di training, poiché implicherebbero una conoscenza aprioristica della natura del traffico, che nel contesto reale è proprio ciò che si intende predire.

Nel complesso, le feature categoriche offrono spunti interessanti ma richiedono una trasformazione o una selezione oculata per poter essere integrate efficacemente

all'interno del processo di modellazione. Alcune di esse, infatti, rischiano di introdurre rumore o complessità non necessaria, mentre altre, se opportunamente trattate, possono contribuire in maniera significativa all'identificazione di comportamenti anomali.

3.2.4 Data Cleaning

In questa fase sono stati utilizzati i risultati dell'analisi per migliorare la qualità del dataset, eliminando feature e record non utili all'addestramento di un modello di machine learning.

Rimozione dei record nulli

Come mostrato dalla tabella 3.3, le colonne `soui`, `doui`, `sco`, `dco`, `smac` e `dmac` hanno il 100% di valori nulli, quindi possono essere semplicemente rimosse dal dataset.

Per le righe restanti con valori nulli, sono così poche che possono essere eliminate senza rischiare di perdere informazioni importanti.

Rimozione delle feature con bassa variazione

Lo studio del coefficiente di variazione ha rivelato una variazione molto bassa delle variabili `stime` e `ltime` (Fig. 3.1), indicando una loro possibile rimozione. Trattandosi di componenti temporali potrebbero essere utili nello sviluppo di un modello *Time Series*, ma anche in quel caso la presenza di ID sequenziali (`pkSeqID` e `seq`) le rende superflue.

Rimozione delle feature ridondanti

Per le feature ridondanti abbiamo fatto riferimento alla tabella 3.8 dei coefficienti di correlazione maggiori di 0.7. Per ogni coppia è stata rimossa una feature, usando come criterio di scelta l'importanza riportata nella tabella 3.9.

La tabella 3.10 mostra quali feature sono state mantenute e quali eliminate in questa fase.

Feature	Stato
dur	Eliminata
sum	Mantenuta
min	Eliminata
mean	Eliminata
max	Mantenuta
stddev	Mantenuta
spkts	Mantenuta
pkts	Eliminata
dpkts	Mantenuta
bytes	Eliminata
dbytes	Mantenuta

Tabella 3.10: Processo di *Feature Selection*

Feature categoriche

Per le feature categoriche, si sarebbe potuto pensare di eliminare `saddr` e `daddr`, ma è stato deciso di mantenerle perché avrebbero potuto nascondere informazioni interessanti.

3.2.5 Data normalization

Alcuni modelli di machine learning mostrano problemi di performance quando le scale dei dati sono molto diverse tra loro, assegnando erroneamente un peso maggiore a numeri più grandi. Per questo motivo è buona pratica, prima di usarlo per allenare un modello, *normalizzare* un dataset, ovvero trasformare tutti i dati in scale simili mantenendone la *forma* in modo da non perdere informazioni importanti.

Le tecniche matematiche per normalizzare i dati sono diverse, per questo lavoro è stata scelta la *z-score normalization*, che applica una trasformazione a un gruppo di valori in modo che abbiano una media pari a 0 e una deviazione standard pari a 1. La formula nello specifico è (3.2.4):

$$z_i = \frac{x_i - \mu}{\sigma} \quad (3.2.4)$$

Dove:

- z_i è il valore normalizzato (Z-score) dell'osservazione x_i
- x_i è il valore originale dell'osservazione
- μ è la media della variabile
- σ è la deviazione standard della variabile

Anche applicando la normalizzazione, alcune feature hanno mantenuto scale troppo grandi per il resto del dataset (ordine delle decine di migliaia contro ordine dei centesimi). Un risultato del genere è facilmente spiegabile tramite lo studio delle statistiche descrittive presente alle tabelle 3.4 e 3.5, che ha evidenziato un'abissale differenza di scala di valori nelle feature `spkts`, `dpkts`, `sbytes`, `dbytes`, `rate`, `srate` e `drate`.

Un modo per gestire questo sbilanciamento è applicare una trasformazione logaritmica ai dati in queste colonne, cosa che ne ridurrebbe sicuramente la scala ma contemporaneamente rischia di "appiattire" la distribuzione dei dati, causando una perdita di informazioni utili per un modello.

Per verificare le conseguenze di questo approccio si è deciso di confrontare la distribuzione delle feature nel dataset originale con quella delle feature a cui è stata applicata la trasformazione; se le due avessero presentato la stessa forma sarebbe stato ragionevole pensare che la scala logaritmica fosse applicabile. Per motivi di efficienza questo studio è stato condotto su un campione ridotto del dataset bilanciato sulla feature `category`. Soprattutto in questa fase il bilanciamento è particolarmente importante, data la presenza massiva di record di attacco `DoS` e `DDoS`, che sono con tutta probabilità proprio i record che causano la differenza di scala.

Una volta calcolate, le distribuzioni sono state rappresentate in istogrammi (Fig. 3.5 a Fig. 3.7), che però si sono dimostrati poco informativi. La presenza di pochi *outlier* ha causato l'apparire della distribuzione nel dataset originale come un'unica linea in prossimità dello 0, rendendo impossibile lo studio della sua forma. Per

ovviare a questo problema i dati sono stati divisi in due *set*, uno contenente il primo 99% dei record e l'altro il restante 1%. Il risultato, mostrato nelle Fig. da 3.8 a 3.10, è estremamente più leggibile, e se confrontato con i grafici in scala logaritmica mostra chiaramente come la forma dei dati si sia mantenuta nonostante la trasformazione.

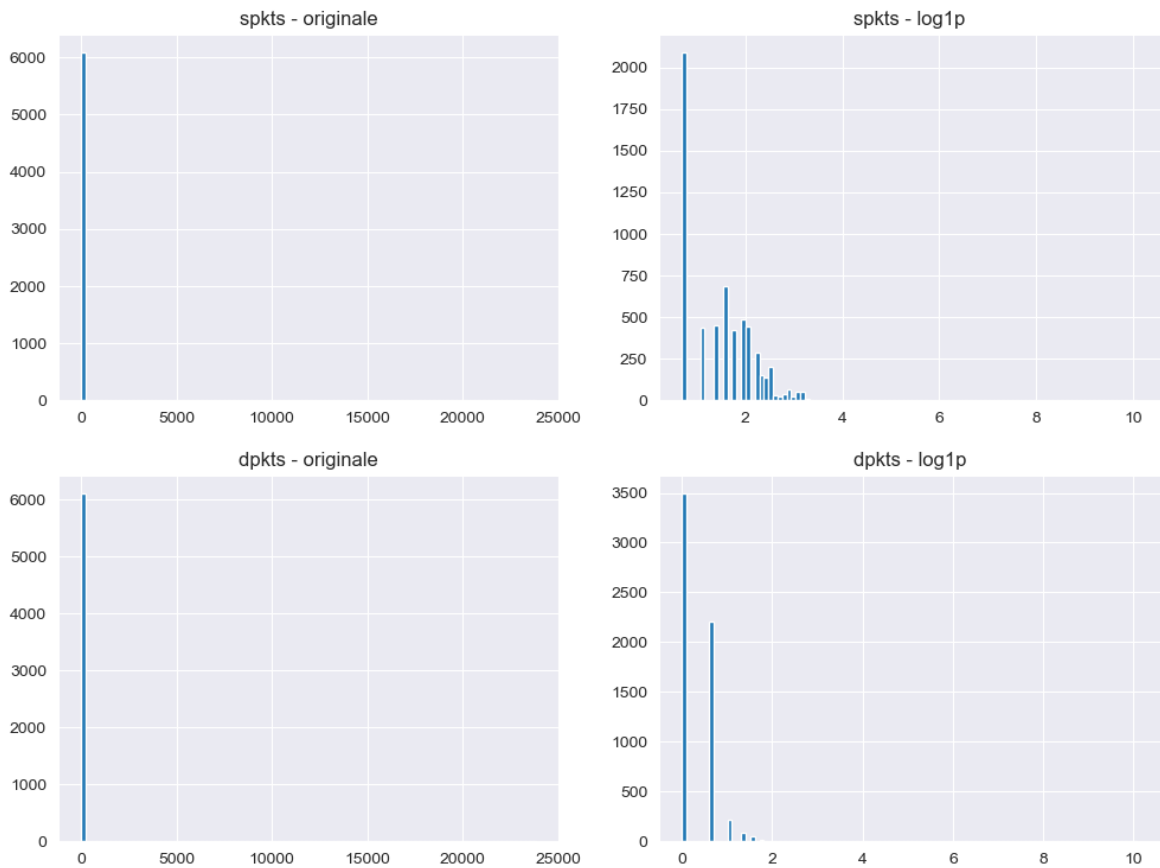


Figura 3.5: Distribuzioni delle feature in scala normale e logaritmica (parte 1).

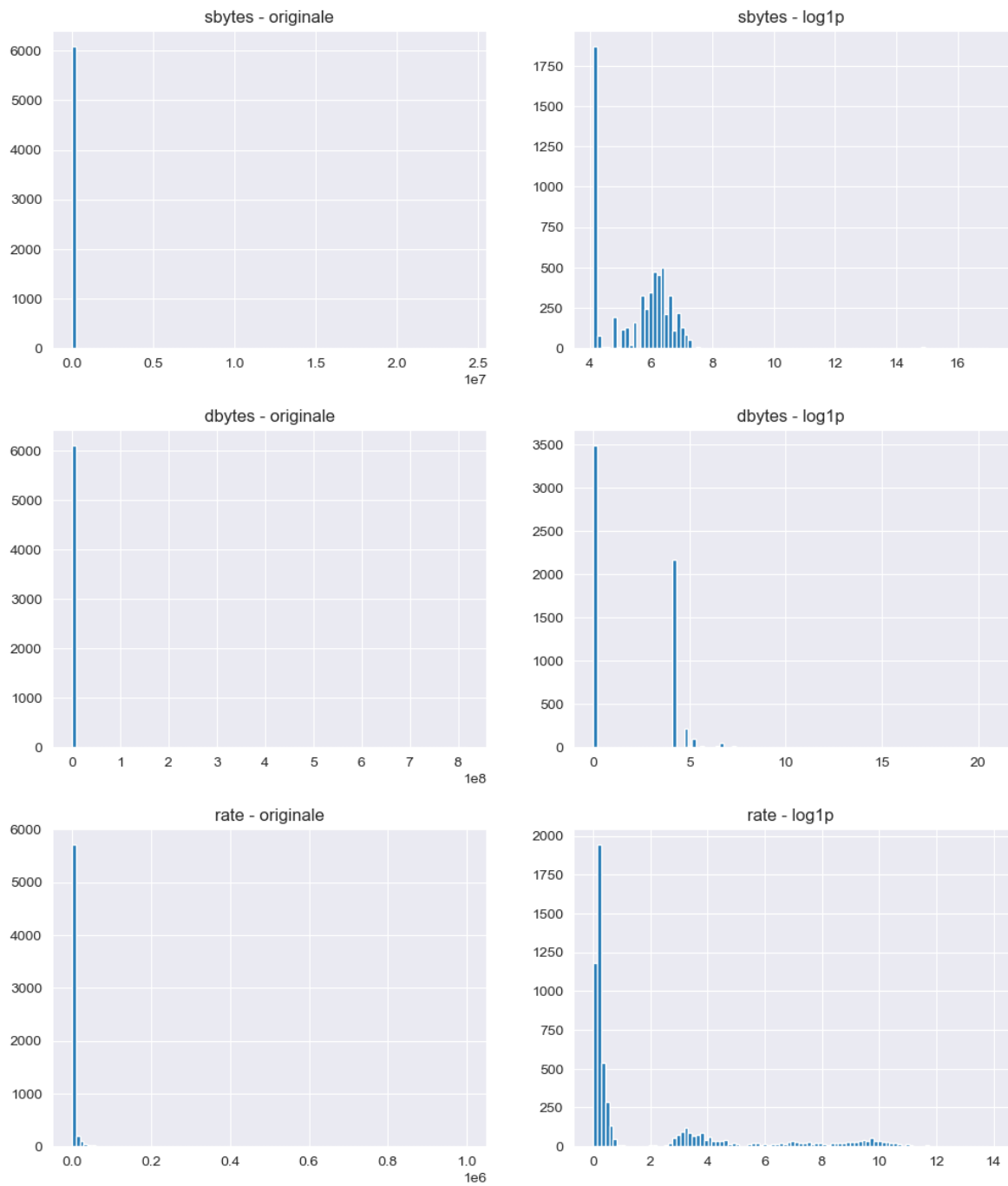


Figura 3.6: Distribuzioni delle feature in scala normale e logaritmica (parte 2).

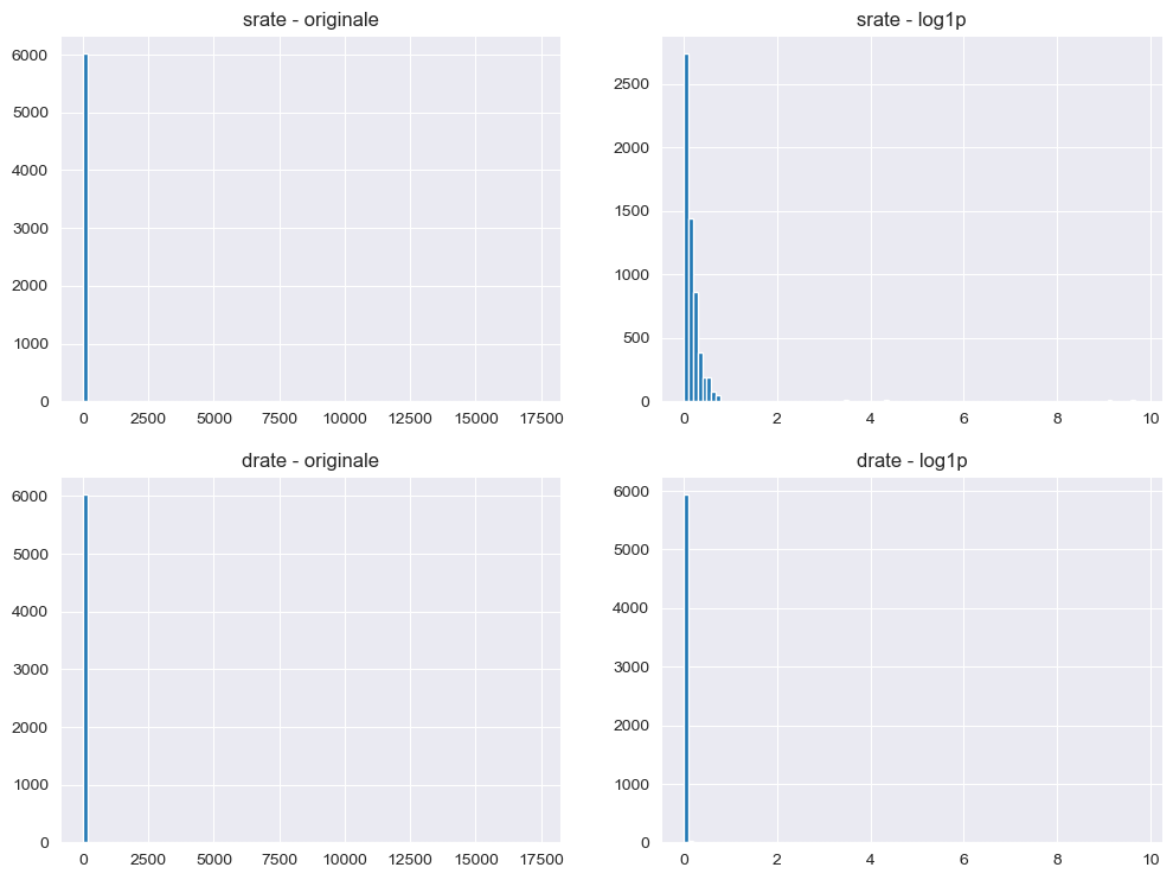


Figura 3.7: Distribuzioni delle feature in scala normale e logaritmica (parte 3).

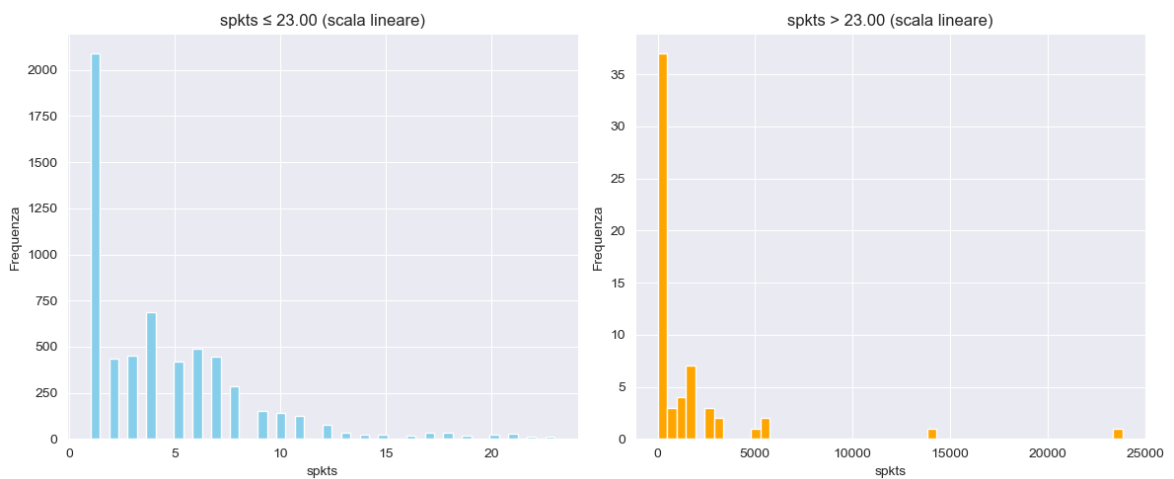


Figura 3.8: Distribuzione della feature in scala normale (diviso, parte 1).

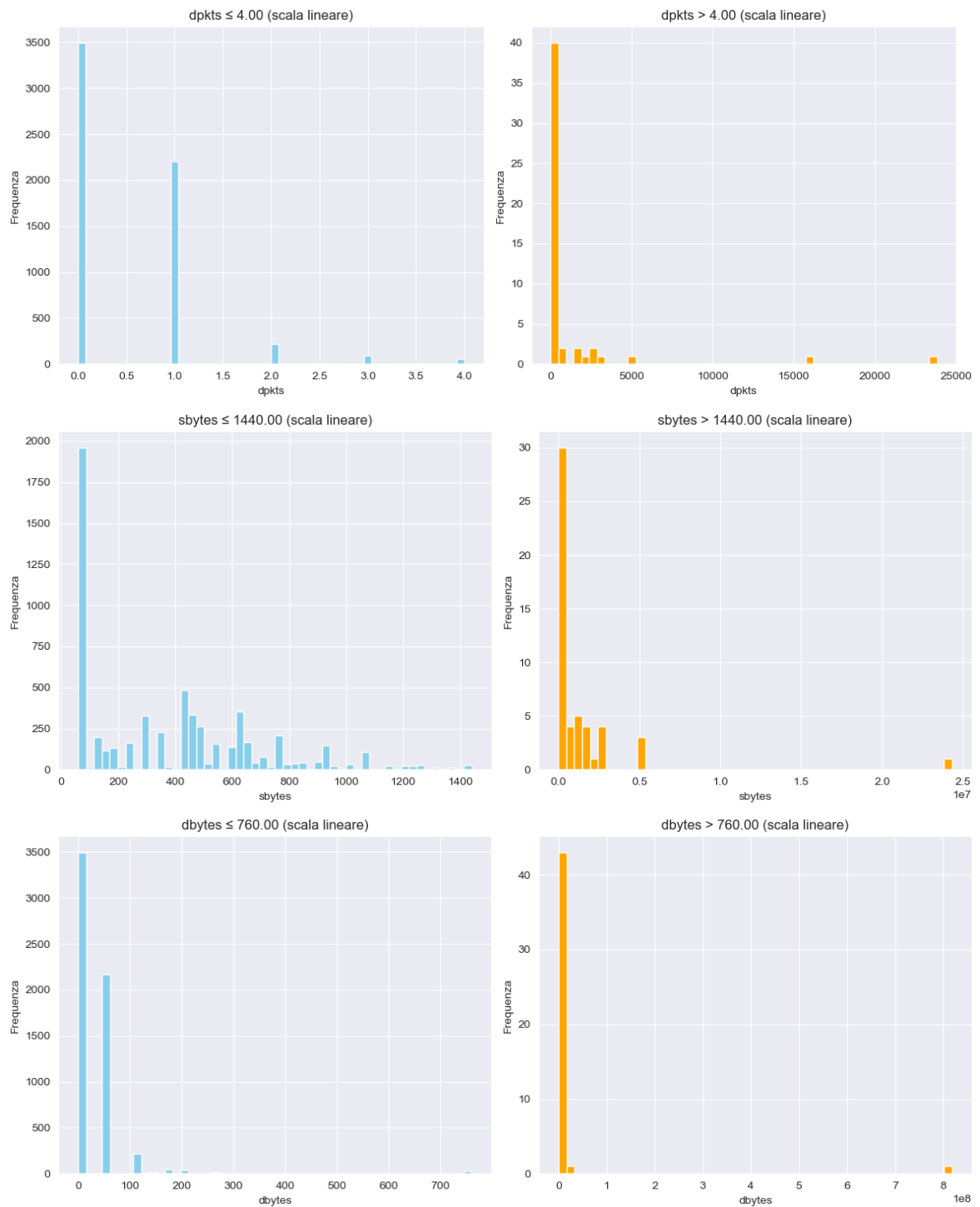


Figura 3.9: Distribuzioni delle feature in scala normale (diviso, parte 2).

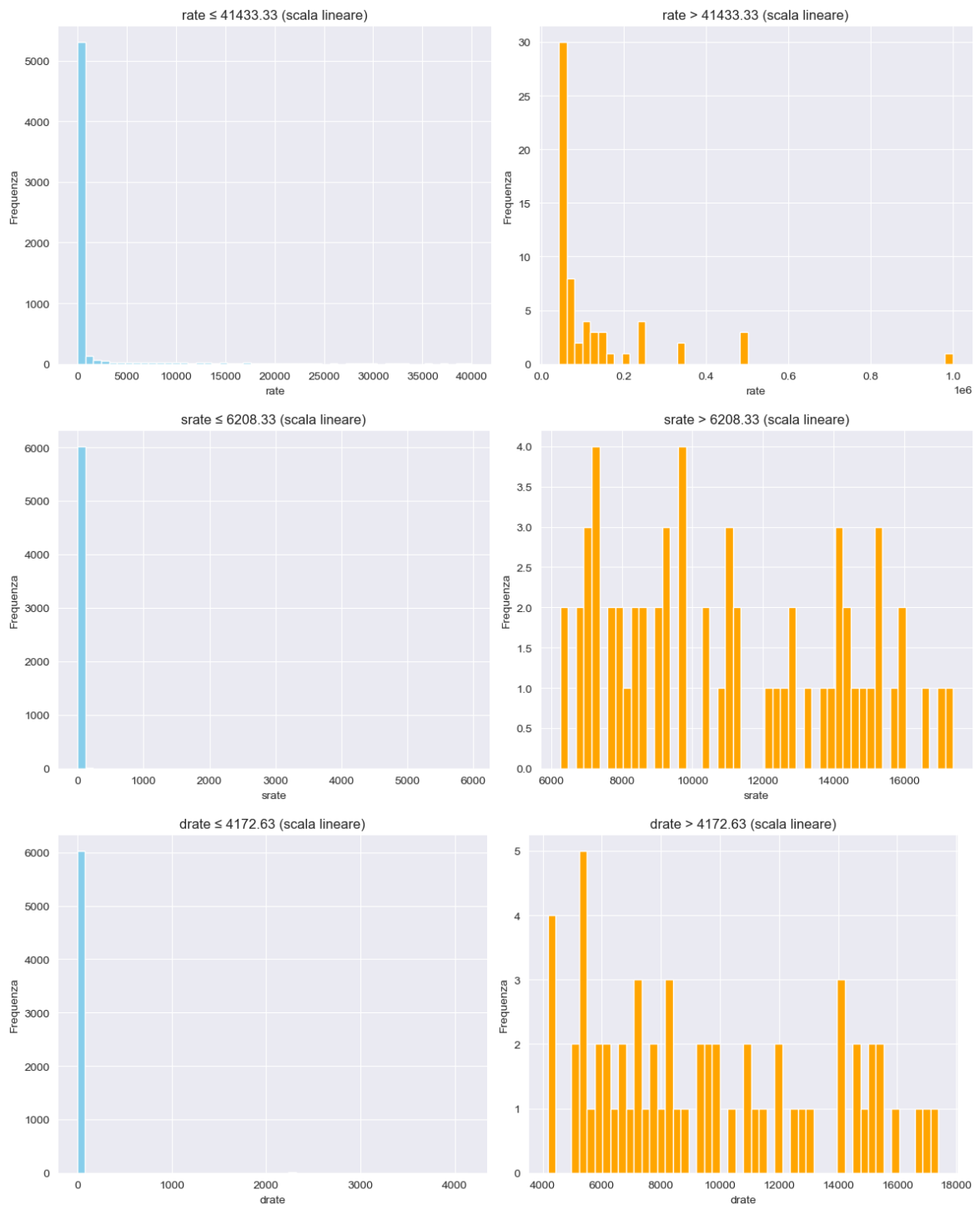


Figura 3.10: Distribuzioni delle feature in scala normale (diviso, parte 3).

Lo studio ha avuto risvolti positivi, quindi si è deciso di applicare la trasformazione logaritmica (in base e) alle feature evidenziate e rinormalizzare.

3.2.6 Data Balancing

Dopo la normalizzazione, l'ultima pratica necessaria per migliorare ancora la qualità del dataset è il *data balancing*. Su un totale di circa 74 milioni di record, in BoT-IoT solo 10.000 rappresentano traffico normale (e anche all'interno di quelli di attacco oltre il 94% appartengono ad attacchi DoS o DDoS). Questo vuol dire che un algoritmo allenato su questi dati che predica sempre che il record presentatogli è di attacco avrà un'accuratezza di oltre il 99%, nonostante nella pratica sia effettivamente inutile.

Non tutti i modelli soffrono di questo *bias*, ma comunque un dataset con un numero bilanciato di record per classe è di qualità migliore rispetto a uno sbilanciato.

Esistono due tecniche per gestire il bilanciamento di un dataset, l'*oversampling*, che consiste nell'aggiunta di record alle classi meno rappresentate, e l'*undersampling*, che consiste nell'eliminazione di record dalle classi più rappresentate. Entrambi gli approcci sono efficaci, ma portano con sé delle conseguenze, il primo introduce nel dataset dati creati sinteticamente che potrebbero creare *pattern* non presenti nella realtà, il secondo riduce la quantità di dati, cosa raramente positiva quando si parla di machine learning.

Nel caso di BoT-IoT, lo sbilanciamento è così grande che queste conseguenze potrebbero essere enormi e causare un danno sostanziale alla bontà del dataset, per cui si è deciso di creare due dataset che saranno usati per l'allenamento dei modelli, uno non bilanciato (quindi quello originale a cui è stato applicato il preprocessing) e uno di 50.000 record, ottenuto dal primo tramite under-sampling delle categorie più rappresentate (ridotte a 10.000 record) e over-sampling delle altre tramite la tecnica SMOTE (*Synthetic Minority Over-sampling Technique*). Il bilanciamento è inoltre stato effettuato sulla feature `category` invece che `attack` per garantire la creazione di un modello capace a riconoscere i tipi di attacco piuttosto che il tipo di traffico.

3.2.7 Data Encoding

Per applicare lo SMOTE, e anche perchè molti modelli lo richiedono, nei dataset finali le feature categoriche sono state convertite in variabili numeriche utilizzando la tecnica del "one-hot encoding", che consiste nella rappresentazione di ogni categoria

come vettore booleano. In questa tecnica viene creata una nuova colonna per ogni categoria, con valore impostato a 1 per indicare la presenza di quella categoria nel record e 0 per la sua assenza.

Per motivi di efficienza le feature che rappresentavano indirizzi IP (`saddr` e `daddr`) e porte di rete (`dport` e `sport`) sono state prima racchiuse in macro categorie (*well-known*, *registered* e *dynamic* per gli IP e *private*, *loopback* e *public* per le porte) e poi soggette a encoding. Questo ha garantito che il dataset rimanesse di grandezza gestibile mantenendo delle eventuali informazioni importanti portate da queste feature.

3.3 Creazione dei modelli

Una volta creati i dataset, si è proceduto con lo sviluppo dei modelli di machine learning. L'obiettivo di questa fase è quello di testare diversi approcci e confrontarli cercando quello che meglio si presta all'*intrusion detection*, verificando contemporaneamente quanto il bilanciamento di un dataset infici sulla performance di un modello.

Ai fini di uno studio generalizzato, si è deciso di testare 4 algoritmi: *Random Forest*, *XGBoost*, *Support Vector Machine* (SVM) e *Multi Layer Perception Classifier* (MLPC).

Ogni modello è stato allenato su entrambi i dataset, divisi in *train set* (80%) e *test set* (20%), e come metriche di valutazione sono state utilizzate *Accuracy*, *Precision*, *Recall* e *F1 Score* (Tab. 2.2).

Random Forest

Random forest è un modello di classificazione che sfrutta un gruppo di *decision trees* per fare previsioni.

Il suo funzionamento è semplice: in fase di training vengono costruiti molti alberi decisionali che a ogni input associano una categoria; l'output della foresta sarà la categoria scelta dalla maggioranza degli alberi.

Per il modello è stata creata una foresta composta da 100 alberi senza profondità massima e con random state uguale a 42.

Come riportato nella tabella 3.11, l'algoritmo ha mostrato risultati eccezionali sia sul dataset ridotto che su quello completo, superando il 99% in tutte le metriche.

Tabella 3.11: Prestazioni del modello Random Forest sui due dataset

Tipo Dataset	Accuracy	Precision	Recall	F1 Score
Sbilanciato	0.9999997	0.9999997	0.9999997	0.9999997
Bilanciato	0.9963	0.9963	0.9963	0.9963

Support Vector Machine

Il *Support Vector Machine* è un algoritmo di apprendimento supervisionato che funziona calcolando, in uno spazio n-dimensionale, un iperpiano che separi record appartenenti a classi diverse. Un nuovo record che deve essere classificato sarà riportato sul piano e gli sarà assegnata una caratteristica in base al lato dell'iperpiano a cui appartiene. Una SVM classica può riconoscere solo dipendenze lineari tra le classi, per individuare quelle non lineari è possibile applicare quello che viene definito "*kernel trick*", ovvero la trasformazione dei dati in uno spazio a dimensione più alta dove dipendenze non lineari lo diventano.

Per il dataset bilanciato è stata creata una SVM con kernel non lineare (*Radial Basis Function*) e parametro di penalità uguale a 1, mentre per quello sbilanciato si è optato per una SVM lineare con parametro di penalità 1 e numero massimo di iterazioni dell'ottimizzatore 10.000.

In questo caso, come osservabile nella tabella 3.12, le prestazioni sono state peggiori, seppur comunque buone, con l'utilizzo del dataset bilanciato rispetto a quello sbilanciato, che ha riportato invece risultati quasi perfetti.

Tabella 3.12: Prestazioni del modello SVM sui due dataset

Tipo Dataset	Accuracy	Precision	Recall	F1 Score
Sbilanciato	0.999947	0.999946	0.999947	0.999941
Bilanciato	0.7766	0.7974	0.7766	0.7781

XGBoost

XGBoost (*Extreme Gradient Boosting*) è una particolare istanza di *gradient boosting*, una tecnica di machine learning che consiste nel miglioramento iterativo di un modello. Esso prevede la creazione successiva di *decision trees*, dove ogni albero è costruito per correggere gli errori di quello precedente; inoltre mette in pratica tecniche di regolazione per evitare *overfitting* e di computazione parallela per migliorarne l'efficienza. È un modello relativamente nuovo, che ha trovato modo di affermarsi come una delle migliori tecniche di classificazione e regressione.

Per questo studio è stato scelto un modello con 100 alberi, ognuno con:

- Profondità massima = 6
- Rate di apprendimento = 0.1
- Evaluation Metric = Multi-Class Log Loss
- Random State = 42

Ancora una volta i risultati dell'addestramento sono stati più che ottimi, superando il 99% in tutte le metriche in entrambi i modelli (Tab. 3.13).

Tabella 3.13: Prestazioni del modello XGBoost sui due dataset

Tipo Dataset	Accuracy	Precision	Recall	F1 Score
Sbilanciato	0.9999961	0.9999961	0.9999961	0.9999961
Bilanciato	0.9962	0.9962	0.9962	0.9962

MLP

Un *Artificial Neural Network* (ANN) è un modello di machine learning ispirato dalla struttura del cervello umano; esso consiste in una struttura di nodi interconnessi, detti *neuroni*. L'informazione passa attraverso il sistema in maniera ordinata, e ogni output prodotto da un neurone influenza l'input degli altri neuroni.

Un *Multi Layer Perceptor* (MLP) è un tipo di ANN formato da più strati di neuroni, spesso associati a funzioni di attivazione (le funzioni input-output) non lineari che gli permettono di identificare pattern complessi all'interno dei dati.

Gli iperparametri utilizzati per il training di questo modello sono:

- 2 hidden layers (100 e 50 neuroni)
- Activation Function - Rectified Linear Unit (relu)
- Solver - Adam
- Max iterations = 100
- Random state = 42

La tabella 3.7 mostra le metriche di performance di questo modello, che superano ancora una volta tutte il 97% per entrambi i dataset.

Tabella 3.14: Prestazioni MLPC sui due dataset

Tipo Dataset	Accuracy	Precision	Recall	F1 Score
Sbilanciato	0.9999882	0.9999882	0.9999882	0.9999882
Bilanciato	0.9750	0.9750	0.9750	0.9750

Balanced Random Forest

In aggiunta a questi modelli, per verificare l'effettiva bontà del dataset completo è stata allenata su di esso anche una *Balanced Random Forest* (BRF), una particolare istanza di Random Forest in cui ogni albero è allenato su un sottoinsieme della classe maggioritaria, in modo da eliminare *bias* e garantire robustezza rispetto all'*overfitting*.

Per continuità, la BRF è stata allenata con gli stessi iperparametri della RF, 100 alberi senza limite di profondità e random state uguale a 42.

La tabella 3.15 mostra i valori di prestazione relativi a questo modello, ancora una volta prossimi alla perfezione.

Tabella 3.15: Prestazioni del Balanced Random Forest

Accuracy	Precision	Recall	F1 Score
0.999638	0.999907	0.999638	0.999745

CAPITOLO 4

Analisi dei risultati

I grafici a istogramma mostrati nelle Fig. 4.1 e 4.2 mettono a confronto gli *F1 Score* dei modelli tra dataset completo e bilanciato, dando una rappresentazione grafica delle loro prestazioni.

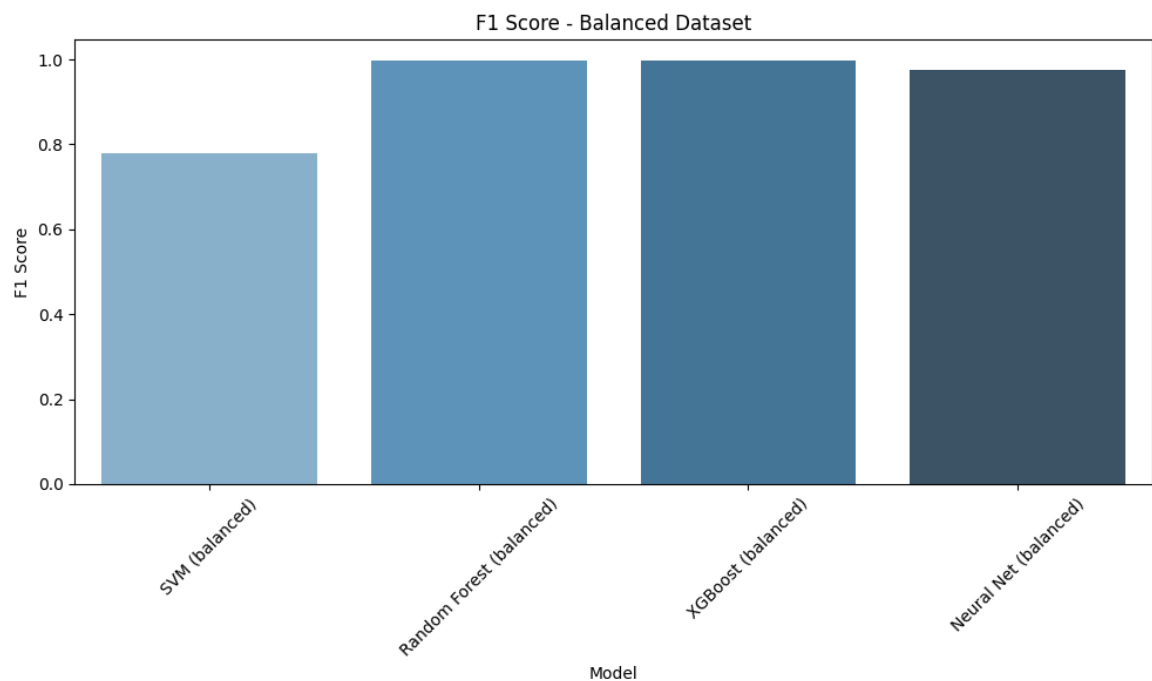


Figura 4.1: F1 Score dei modelli allenati su dataset bilanciato

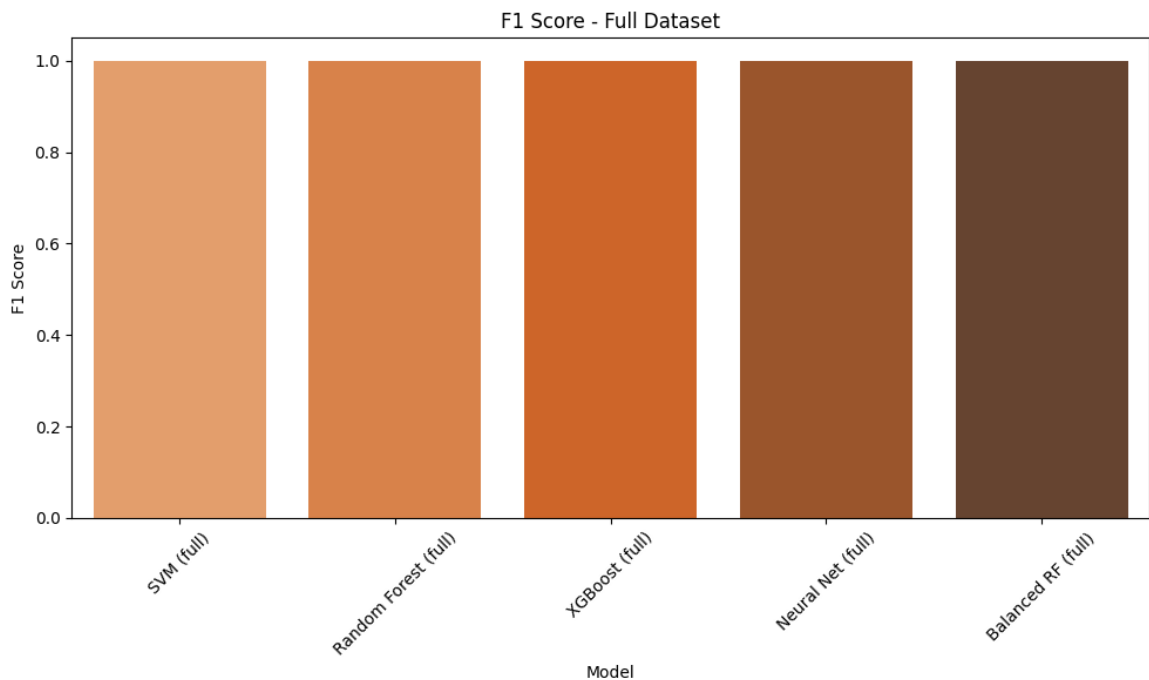


Figura 4.2: F1 Score dei modelli allenati su dataset completo

Oltre ad essere estremamente promettenti, questi risultati sono anche in linea con quelli di altri lavori presenti nello stato dell'arte, cosa che in un primo momento può apparire positiva ma che in realtà offre degli importanti spunti di riflessione sullo stato degli IDS in ambito IOT.

Dei livelli di accuratezza così alti, infatti, spesso indicano che i modelli sono andati incontro ad *overfitting*, e quindi, nonostante in fase di training abbiano ricevuto ottimi risultati, non siano utilizzabili in un ambiente reale. Un comportamento del genere è aspettato nei modelli addestrati sul dataset completo a causa del suo enorme sbilanciamento, ma il fatto che anche gli altri (fatta eccezione per il SVM) abbiano riportato risultati simili fa pensare che il problema non sia nella *quantità*, bensì nella *qualità* del dataset, che contiene forse troppe poche classi d'attacco o troppi pochi tipi di traffico normale.

Ovviamente rimane plausibile che gli algoritmi sviluppati in questo e altri lavori siano effettivamente efficaci in un ambiente reale, ma nel caso in cui si rivelino deboli risultati del genere suggeriscono che l'attenzione non debba essere rivolta tanto a cercare modelli migliori (anche perchè se volessimo rifarci alle metriche classiche i modelli presenti nello stato dell'arte risultano già praticamente perfetti), bensì

dati migliori e aggiornati, magari combinati con tecniche di "online learning" che si possano adattare anche ad attacchi "zero-day" man mano che vengono individuati.

CAPITOLO 5

Conclusioni

L'obiettivo di questo lavoro era di affrontare lo sviluppo di un Intrusion Detection System specifico per l'IoT con gli strumenti disponibili, verificando quanto fossero riproducibili i risultati ottenuti nello stato dell'arte e confrontando diversi modelli di machine learning per scoprire quale approccio si prestasse meglio a questa task.

Per farlo è stato utilizzato uno dei dataset più riconosciuti in letteratura, BoT-IoT, opportunamente preprocessato per essere reso il più adatto possibile all'addestramento di un modello. Una volta preprocessato, quest'ultimo è stato usato per allenare diversi modelli di machine learning, per verificare quale fosse il più efficiente nell'ambito dell'intrusion detection.

Lo studio è iniziato con la definizione di due *domande di ricerca*, a cui adesso daremo risposta in base ai risultati ottenuti.

Risposte alle domande di ricerca

Domanda 1

Quali sono le prestazioni raggiungibili da modelli di Machine Learning per l’Intrusion Detection in ambiente IoT?

Con metriche di valutazione prossime a 1, tutti i modelli testati (Random Forest, XGBoost, SVM, MLP) hanno raggiunto risultati comparabili con lo stato dell’arte. Questi risultati mostrano che, con un preprocessing adeguato, è possibile ottenere alte prestazioni anche su dataset complessi come BoT-IoT. Tuttavia, le metriche classiche potrebbero non essere sufficienti a garantire l’efficacia in ambienti reali.

Domanda 2

Quali sono i punti critici dello sviluppo di tali modelli?

Durante lo sviluppo si sono evidenziate alcune criticità rilevanti:

- lo **sbilanciamento delle classi** nei dataset, che ha influenzato negativamente i modelli in fase di test;
- la **scarsa generalizzazione** a causa di feature troppo specifiche o simulate;
- l’assenza di **metriche post-rilascio** che permettano una valutazione continua in ambienti reali.

Questi elementi indicano che, oltre alla fase di addestramento, è cruciale progettare una pipeline di valutazione continua e contestuale.

Considerazioni finali

Piuttosto che concentrarsi sui *modelli*, questo studio pone l’attenzione sulla limitazione dei *dati* disponibili oggi in ambito di Intrusion Detection per IoT, che forse necessitano di una maggiore qualità per poter essere viabili nell’addestramento di modelli efficaci.

A questo proposito appare evidente l'importanza per i lavori futuri di utilizzare per la validazione dei modelli operanti in questo ambito anche metriche di performance in ambienti reali, soprattutto vista la natura sensibile del problema che devono affrontare.

Bibliografia

- [1] IoT Analytics, “Number of connected iot devices (global forecast),” 2024. [Online]. Available: <https://iot-analytics.com/number-connected-iot-devices> (Citato alle pagine iii e 5)
- [2] A. Gluhak, K. Moessner, T. Popovic, S. He, and S. Krco, “Architecture,” in *Springer Handbook of Internet of Things*, S. Ziegler, R. Radócz, A. Quesada Rodriguez, and S. N. Matheu Garcia, Eds. Springer, Cham, 2024, pp. 93–109. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-39650-2_5 (Citato alle pagine iii e 6)
- [3] S. Gatlan, “Iot attacks escalating with a 217.5% increase in volume,” BleepingComputer (online), Mar. 2019. [Online]. Available: <https://www.bleepingcomputer.com/news/security/iot-attacks-escalating-with-a-2175-percent-increase-in-volume> (Citato alle pagine iii e 6)
- [4] A. Kumar, “What is intrusion detection and prevention systems (idps)?” DevSecOps Now (online), n.d. [Online]. Available: <https://www.devsecopsnow.com/what-is-intrusion-detection-and-prevention-systems-idps/> (Citato alle pagine iii e 8)

- [5] "Difference between nids and hids," <https://www.techyv.com/questions/difference-between-nids-and-hids-and-how-do-they-help/>. (Citato alle pagine iii e 8)
- [6] "Supervised learning," <https://botpenguin.com/glossary/supervised-learning>. (Citato alle pagine iii e 10)
- [7] "Understanding machine learning basics," <https://aigen023.blogspot.com/2024/09/understanding-machine-learning-basics.html>. (Citato alle pagine iii e 11)
- [8] "Types of machine learning," <https://techvidvan.com/tutorials/types-of-machine-learning/>. (Citato alle pagine iii e 12)
- [9] M. M. Rahman, S. Al Shakil, and M. R. Mustakim, "A survey on intrusion detection system in iot networks," *Cyber Security and Applications*, vol. 3, p. 100082, 2025. (Citato alle pagine v e 18)
- [10] "What is the internet of things (iot)?" [Online]. Available: <https://www.ibm.com/topics/internet-of-things> (Citato a pagina 4)
- [11] R. Ande, B. Adebisi, M. Hammoudeh, and J. Saleem, "Internet of things: Evolution and technologies from a security perspective," *Sustainable Cities and Society*, vol. 54, p. 101728, 2020. (Citato a pagina 4)
- [12] P. Licata, "Iot, nel 2025 il giro d'affari da mille miliardi. ai e 5g i driver della crescita," *Corriere Comunicazioni* (online), Jan. 2025, accessed: 2025-06-26. [Online]. Available: <https://www.corrierecomunicazioni.it/digital-economy/iot-nel-2025-il-giro-daffari-da-mille-miliardi-ai-e-5g-i-driver-della-crescita/> (Citato a pagina 5)
- [13] Google, "Overview of internet of things," Online, 2017. (Citato a pagina 5)
- [14] G. Lindemulder and M. Kosinski, "Che cos'è la cybersecurity?" IBM Italia (online), Aug. 2024. [Online]. Available: <https://www.ibm.com/it-it/topics/cybersecurity> (Citato a pagina 7)

-
- [15] IBM, "Che cos'è un attacco informatico?" IBM Think (online), Aug. 2021. [Online]. Available: <https://www.ibm.com/it-it/think/topics/cyber-attack> (Citato a pagina 7)
- [16] A. Mahendiran, A. Rajesh, and S. Karthik, "Intrusion detection and prevention system: Technologies and challenges," *International Journal of Applied Engineering Research*, vol. 10, no. 87, pp. 1–12, 2015. [Online]. Available: https://www.researchgate.net/publication/287208734_Intrusion_Detection_and_Prevention_System_Tchnologies_and_Challenges (Citato a pagina 7)
- [17] L. Arnaboldi and C. Morisset, "A review of intrusion detection systems and their evaluation in the iot," *arXiv preprint arXiv:2105.08096*, 2021. (Citato alle pagine 7 e 9)
- [18] P. Mell, V. Hu, R. Lippmann, J. Haines, and M. Zissman, "An overview of issues in testing intrusion detection systems," 2003. (Citato a pagina 7)
- [19] "Intelligenza artificiale," Treccani - Enciclopedia (online). [Online]. Available: <https://www.treccani.it/enciclopedia/intelligenza-artificiale/> (Citato a pagina 9)
- [20] E. Alpaydin, *Machine Learning*. MIT Press, 2021. (Citato a pagina 9)
- [21] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015. (Citato a pagina 9)
- [22] Y. Rathod, "A survey of machine learning techniques for artificial intelligence," *International Journal of Computer Techniques*, vol. 11, 07 2024. (Citato a pagina 10)
- [23] L. Tawalbeh, F. Muheidat, M. Tawalbeh, and M. Quwaider, "Iot privacy and security: Challenges and solutions," *Applied Sciences*, vol. 10, no. 12, p. 4102, 2020. (Citato a pagina 11)
- [24] M. Litoussi, N. Kannouf, K. El Makkaoui, A. Ezzati, and M. Fartitchou, "Iot security: challenges and countermeasures," *Procedia Computer Science*, vol. 177, pp. 503–508, 2020. (Citato a pagina 11)

- [25] E. Ashraf, N. Areed, H. Salem Marie, E. Abdelhady, and A. Farouk, "Iot based intrusion detection systems from the perspective of machine and deep learning: A survey and comparative study," *Delta University Scientific Journal*, vol. 5, pp. 367–386, 12 2022. (Citato alle pagine 12, 13 e 15)
- [26] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset," *arXiv preprint arXiv:1811.00701*, 2018. (Citato alle pagine 13, 14, 20 e 21)
- [27] N. Moustafa, "The bot-iot dataset," *IEEE Dataport*, 2019. (Citato alle pagine 13 e 14)
- [28] S. J. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. K. Chan, "Kdd cup 1999 data," UCI KDD Archive, Tech. Rep., 1999. (Citato alle pagine 13 e 14)
- [29] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the kdd cup 99 data set," *IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009. (Citato alle pagine 13 e 14)
- [30] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023. (Citato alle pagine 13 e 14)
- [31] M. A. Ferrag, L. Shu, A. Derhab, J. Zhang, L. Maglaras, and H. Janicke, "Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications," *arXiv preprint arXiv:2207.12346*, 2022. (Citato alle pagine 13 e 14)
- [32] J. Mathews, P. Chatterjee, and S. Banik, "Coap-dos: An iot network intrusion dataset," *arXiv preprint arXiv:2210.03313*, 2022. (Citato alle pagine 13 e 14)
- [33] M. Rabbani, J. Gui, F. Nejati, Z. Zhou, A. Kaniyamattam, M. Mirani, G. Piya, I. Opushnyev, R. Lu, and A. A. Ghorbani, "Device identification and anomaly detection in iot environments (cic iot-diad)," *IEEE DataPort*, 2024. (Citato alle pagine 13 e 14)

-
- [34] M. Koppula and L. J. M. I. Joseph, "A real-world dataset "idsiot2024" for machine learning/deep learning based cyber security," *IEEE DataPort*, 2025. (Citato alle pagine 13 e 14)
- [35] O. Belarbi, T. Spyridopoulos, E. Anthi, O. Rana, P. Carnelli, and A. Khan, "Gotham dataset 2025: A reproducible large-scale iot network dataset for intrusion detection and security research," *arXiv preprint arXiv:2505.09999*, 2025. (Citato alle pagine 13 e 14)
- [36] D. Herzalla, W. T. Lunardi, and M. A. Lopez, "Tii-ssrc-23 dataset: Typological exploration of diverse traffic patterns for intrusion detection," *arXiv preprint arXiv:2303.09211*, 2023. (Citato alle pagine 13 e 14)
- [37] J. Kim, J. Kim, H. L. T. Thu, and H. Kim, "Long short term memory recurrent neural network classifier for intrusion detection," in *2016 international conference on platform technology and service (PlatCon)*. IEEE, 2016, pp. 1–5. (Citato a pagina 14)
- [38] X. Xu and Y. Chen, "Automl-based intrusion detection for iot networks using kddcup99," *Future Internet*, 2023. (Citato alle pagine 16 e 18)
- [39] M.-D. Ngo and T. Nguyen, "Machine learning-based intrusion detection using feature selection and extraction," *Sensors*, vol. 23, 2023. (Citato alle pagine 16 e 18)
- [40] E. Viegas, L. Oliveira, and S. Barbon Jr., "Toward trustworthy evaluation schemes for network-based intrusion detection systems," *Computers & Security*, 2023. (Citato alle pagine 16 e 18)
- [41] M. Zakariah and P. Thomas, "Adaptive synthetic sampling technique using lstm and cnn for intrusion detection," *IEEE Access*, 2023. (Citato alle pagine 16 e 18)
- [42] Y. Gu and Z. Lu, "A hybrid svm-naive bayes feature embedding for intrusion detection in iot," *Journal of Network and Computer Applications*, 2021. (Citato alle pagine 16 e 18)

- [43] L. Guo and L. Huang, "Time-varying chaos particle swarm optimization with support vector machines for intrusion detection," *Neurocomputing*, vol. 218, pp. 255–262, 2016. (Citato alle pagine 17 e 18)
- [44] M. Kabir and R. Islam, "Oa-ls-svm for efficient intrusion detection," *Journal of Computer and System Sciences*, 2018. (Citato alle pagine 17 e 18)
- [45] S. Singh, N. Kumar, and D. Juneja, "An efficient intrusion detection system using online sequential elm and network profiling," *Computers & Electrical Engineering*, vol. 47, pp. 275–285, 2015. (Citato alle pagine 17 e 18)
- [46] "Bot-iot dataset," <https://www.kaggle.com/datasets/vigneshvenkateswaran/bot-iot>. (Citato a pagina 20)

Ringraziamenti

Ringrazio la mia famiglia, che mi ha sostenuto e guidato tutta la vita.

Ringrazio Angelo e Gabriele, che non mi hanno mai voltato le spalle e mi hanno insegnato il vero significato di amicizia.

Ringrazio Marco, Michele e Frank, con cui ho condiviso i momenti migliori e peggiori di questi tre anni, e spero di tanti anni a venire.

Ringrazio Bruno e la sua famiglia, che è anche la mia famiglia.

Ringrazio i ragazzi dell'università, delle persone fantastiche che mi hanno permesso di non essere mai solo nel luogo in cui la solitudine è il nemico più grande.

Ringrazio i ragazzi di Torre, che tutto sommato non sono tanto male.

Infine ringrazio Giovanni, alla cui pazienza e altruismo devo una buona metà dei miei risultati accademici.

Grazie Prof. Palomba per la guida in questo viaggio.

Questa tesi ha contribuito a piantare un albero in Tanzania tramite il progetto Treedom

<https://www.treedom.net/it/user/sesalab/event/sesa-random-forest>